

InsetGAN for Full-Body Human Image Generation

Joint GAN Compositing, Latent-Space Inversion, and System Implementation

Adebanji Oluwatimileyin Adelowo

2026

Contents

1	Introduction and Motivation	3
1.1	The full-body human generation problem	3
1.2	Project context	4
1.3	What this document covers	4
2	Problem Definition	5
2.1	Formal statement	5
2.2	Why a single conditional generator is the wrong tool	5
2.3	The compositing-of-specialists alternative	5
2.4	The role of inversion	6
3	Mathematical and Terminological Foundations	6
3.1	The adversarial objective	6
3.2	Latent spaces: $\mathcal{Z}$, $\mathcal{W}$, and $\mathcal{W}+$	7
3.3	Style modulation: AdaIN and its successor	8
3.4	The truncation trick	9
3.5	GAN inversion as an optimisation problem	9
3.6	Perceptual and identity losses	10
4	Classical and Foundational Generative Approaches	11
4.1	Variational autoencoders	11
4.2	Early GANs and their failure modes	11
4.3	Why StyleGAN was a discontinuity	12
5	Research Evolution: From DCGAN to Multi-GAN Compositing	12
5.1	Stage 1: architectural stabilisation (2015–2018)	12
5.2	Stage 2: style-based generation (2019–2020)	12
5.3	Stage 3: domain specialisation (2022)	13
5.4	Stage 4: the inversion problem and encoder-based solutions	13
5.5	Stage 5: moving the generator instead of the code	14
5.6	Stage 6: compositing multiple specialist GANs	14
6	StyleGAN2 in Detail	15
6.1	The mapping network	15

6.2	The synthesis network	15
6.3	Weight demodulation: derivation	15
6.4	The truncation trick in the synthesis network	17
6.5	Style mixing and mixing regularisation	17
7	The ReStyle Encoder: Iterative Residual Inversion	17
7.1	The limitation of single-pass encoders	17
7.2	The iterative scheme	18
7.3	Why iteration helps	18
7.4	Training objective	19
7.5	The implementation in this repository	19
7.6	A note on the portrait-aspect decoder	20
8	Pivotal Tuning Inversion	20
8.1	Motivation	20
8.2	Stage 1: inversion to the pivot	20
8.3	Stage 2: pivotal tuning	21
8.4	Stage 3: locality regularisation	21
8.5	PTI in this repository	22
9	InsetGAN: The Joint Optimisation Objective	22
9.1	Setup and notation	23
9.2	The master problem	23
9.3	Loss term 1: coarse appearance, $\mathcal{L}_A$	23
9.4	Loss term 2: boundary blending, $\mathcal{L}_B$	24
9.5	Loss term 3: latent regularisation, $\mathcal{L}_R$	24
9.6	Loss terms 4 and 5: anchoring the two halves	25
9.7	The four application modes	25
9.8	The alternating optimisation schedule	26
9.9	Why gradients can flow through two frozen, independently trained generators	26
9.10	Reported cost and quality	27
10	System Architecture and Code Implementation	27
10.1	Repository layout	27
10.2	The generation pipeline end to end	28
10.3	Step 1: environment and model acquisition	28
10.4	Step 2: face alignment	29
10.5	Step 3: ReStyle inversion	29
10.6	Step 4: the InsetGAN module	30
10.6.1	The loss terms as implemented	30
10.6.2	The $\mathcal{W}+$ decomposition in code	32
10.6.3	The moving crop	32
10.6.4	The three-stage schedule	33
10.6.5	Optimisers and learning-rate schedule	35
10.6.6	Composition and output	35
10.6.7	The CLI entry point	36
10.7	Step 5: adapting InsetGAN to a real user photo	36
10.8	Step 6: the FastAPI serving layer	37

10.8.1 Security finding: hardcoded credentials committed to the repository	39
10.9 Step 7: the iOS client and the Firebase relay	40
11 Experiments and Demonstrated Capabilities	41
11.1 Unconditional full-body generation	41
11.2 Attribute editing via latent directions	41
11.3 Style mixing	43
11.4 InsetGAN joint optimisation	43
11.5 The full personalised pipeline	43
11.6 Auxiliary data-cleaning experiments	43
12 Limitations and Failure Cases	44
12.1 Identity fidelity is bounded by the latent space	44
12.2 Seam and blending artefacts	44
12.3 Artefacts inherited from the body generator	45
12.4 Dataset bias	45
12.5 Inference latency	45
12.6 Serving fragility	45
12.7 Security and privacy defects	46
12.8 Reproducibility caveats	46
13 Future Research Directions	47
13.1 The GAN-to-diffusion transition, as a case study	47
13.2 Extensions within the GAN framework	48
13.3 3D-aware generation	48
13.4 Video and temporal extension	48
13.5 On-device inference	49
13.6 Evaluation methodology	49
14 References	49
14.1 Software and repositories	52

1 Introduction and Motivation

1.1 The full-body human generation problem

Unconditional image synthesis with generative adversarial networks reached photorealism first on domains that are *statistically narrow and geometrically aligned*. Human faces are the canonical example: the FFHQ dataset is aligned on eye positions, every image contains exactly one object class in a near-canonical pose, and the intra-class variability is dominated by appearance (skin tone, hair, illumination) rather than articulation. StyleGAN2 trained on FFHQ at 1024×1024 produces images that most human observers cannot reliably distinguish from photographs.

Full-body humans are a categorically harder domain. A standing person is an articulated body with roughly thirty degrees of freedom in pose alone, wrapped in clothing whose geometry and texture are themselves generative processes, attached to a head that carries all the perceptual weight of a face, and photographed against arbitrary backgrounds. A generator with a fixed capacity budget must spread that capacity across all of these factors. The empirical consequence, documented by Fröhstück et al. [15], is that a state-of-the-art StyleGAN2 trained on eighty thousand curated

full-body photographs at 1024×1024 produces plausible global body proportions and clothing but degrades exactly where human observers look first: faces, hands, and feet. The face occupies perhaps two percent of the pixels of a full-body portrait but consumes a disproportionate share of the observer’s attention, and it is the region where the generator has the least effective training signal per unit of perceptual importance.

This document is a technical and research account of **InsetGAN** — a multi-generator joint optimisation framework that resolves this tension not by training a bigger model, but by *composing several independently pretrained specialists* — and of the system built around it in this repository: a face-personalised, full-body human generator with an inference server and an iOS client.

1.2 Project context

The system documented here, **Whole-Body GAN Generator**, was built during a machine learning internship at Fellowship.AI (May–August 2022). Its stated product goal was concrete: a user photographs their own face with an iPhone, and the system returns a photorealistic full-body image of a person who looks like them, wearing generated clothing, at 1024-pixel resolution.

Delivering that goal required assembling four distinct research contributions into one pipeline:

1. **StyleGAN-Human** [13] supplies the body generator: a StyleGAN2 trained on a large, carefully re-aligned corpus of full-body human photographs.
2. **ReStyle** [22] supplies the *encoder*: it maps the user’s real face photograph into the $\mathcal{W}+$ latent space of a StyleGAN2-FFHQ face generator through iterative residual refinement.
3. **Pivotal Tuning Inversion (PTI)** [23] supplies a higher-fidelity but slower alternative to the encoder, trading inference time for reconstruction accuracy by fine-tuning the generator itself around the inverted code.
4. **InsetGAN** [15] supplies the compositing mechanism: it jointly optimises the face latent code and the body latent code so that the face, pasted into the body canvas by a plain rectangular copy, produces no visible seam and no appearance mismatch.

Around these sits an engineering layer that is a genuine part of the artefact and not an afterthought: a FastAPI inference server hosted inside a Google Colab GPU runtime, exposed to the public internet through an ngrok tunnel, and a Swift/UIKit iOS client that transfers images through Firebase Storage.

1.3 What this document covers

Sections 2 and 3 define the problem formally and lay out the mathematical vocabulary — the adversarial objective, the three latent spaces $\mathcal{Z}$, $\mathcal{W}$, $\mathcal{W}+$, style modulation, and the GAN inversion problem. Section 4 gives brief context on the pre-StyleGAN generative landscape, and Section 5 traces the research lineage from DCGAN through to multi-GAN compositing. Sections 6 through 9 are the technical core: StyleGAN2’s architecture with a derivation of weight demodulation, ReStyle’s iterative residual encoder, PTI’s two-stage generator fine-tuning, and InsetGAN’s joint optimisation objective derived from the primary source. Section 10 walks the actual code in this repository and connects every engineering decision back to the mathematics. Sections 11 through 13 cover the demonstrated capabilities, the honest failure modes, and the trajectory that led this project to be superseded by a diffusion-based successor in 2026.

2 Problem Definition

2.1 Formal statement

Let x_{face} be a real photograph containing a user’s face. We want to produce a full-body image $I \in \mathbb{R}^{3 \times H \times W}$ with $H = 1024$ such that:

- **(P1) Realism.** I lies close to the manifold of real full-body human photographs; a human observer should not immediately classify it as synthetic.
- **(P2) Identity.** The face region of I is recognisably the person in x_{face} .
- **(P3) Coherence.** The face region and the body region agree in skin tone, illumination direction, colour temperature, hair continuation, gender presentation, and apparent age.
- **(P4) Seamlessness.** No visible boundary artefact exists where the face region meets the body region.
- **(P5) Diversity.** Different runs, or different random seeds, produce genuinely different bodies for the same input face.

2.2 Why a single conditional generator is the wrong tool

The most direct approach would be a single generator G conditioned on the face: $I = G(z, c(x_{\text{face}}))$, where c is some face-identity encoder. Four difficulties make this unattractive in the 2022 setting.

Capacity dilution. A single network must model pose, body shape, garment geometry, garment texture, hair, background, *and* facial identity. Empirically the face loses. Fröhstück et al. observe directly that their full-body StyleGAN2 exhibits “artifacts ... most visibly in extremities and faces” [15]. The face region in a 1024×1024 full-body portrait is a crop of roughly 150×150 pixels; the generator effectively models faces at a resolution far below what a dedicated face GAN sees.

Data scarcity at the intersection. Aligned face datasets are enormous (FFHQ: 70{,}000 images at 1024^2 , aligned to sub-pixel eye positions). Full-body datasets at comparable resolution are rare and expensive: the InsetGAN authors had to purchase a proprietary corpus of 100{,}718 photographs and filter it down to 83{,}972 usable images. No dataset in 2022 combined FFHQ-grade facial detail with full-body framing at scale.

Conditional architectures lagged in resolution. As the InsetGAN paper notes, “many conditional architectures are not able to handle the same high resolution (1024×1024 px) of unconditional StyleGAN” [15]. Training a conditional model at that resolution from scratch was a multi-GPU-month proposition with no guarantee of matching the unconditional baseline.

Retraining destroys reusable assets. StyleGAN2-FFHQ and StyleGAN-Human already exist as frozen, high-quality checkpoints. A conditional retraining throws away that investment.

2.3 The compositing-of-specialists alternative

InsetGAN inverts the framing. Instead of asking one generator to be good everywhere, it asks two generators — each already excellent in its own domain — to *agree with each other in a narrow region of overlap*.

Concretely: let G_A be the canvas generator (the full-body model) and G_B the inset generator (the face model). Both are frozen. Let $\mathcal{B}(\cdot)$ be a bounding-box crop operator that, given a full-body image, returns the face region located by a face detector. The compositing operation is a literal

pixel replacement,

$$I = \text{paste}(G_A(w_A), G_B(w_B), \mathcal{B}(G_A(w_A))),$$

which introduces a seam unless $G_B(w_B)$ happens to agree with $\mathcal{B}(G_A(w_A))$ at the boundary. InsetGAN’s contribution is to *make it happen* by searching the two latent spaces jointly:

$$\min_{w_A, w_B} \int_{\Omega} \mathcal{L}(G_A(w_A), G_B(w_B)), \quad \Omega := \mathcal{B}(G_A(w_A)).$$

This is Equation (1) of the InsetGAN paper [15]. Two features of it are worth flagging immediately, because they drive everything in Section 9.

First, the domain of integration Ω *depends on the optimisation variable* w_A : moving the body latent moves the face, which moves the bounding box, which changes the region over which the loss is computed. This is a moving-boundary problem, and it is why the optimisation must be alternating rather than simultaneous.

Second, the loss couples two networks that were never trained together and share no weights. Gradients flow through both, but only into the latent inputs, never into the parameters. The generators are used purely as differentiable, frozen decoders — fixed nonlinear maps whose Jacobians define how appearance responds to latent perturbation.

2.4 The role of inversion

Properties (P1), (P3), (P4) and (P5) are addressed by the joint optimisation. Property (P2) — identity — requires something else: the user’s real face must first be represented as a latent code w_B that the frozen face generator can decode back into approximately that face. That is the **GAN inversion** problem, and the two solutions this repository implements, ReStyle (fast, encoder-based) and PTI (slow, optimisation-plus-fine-tuning), are covered in Sections 7 and 8.

The full pipeline therefore reads:

$$x_{\text{face}} \xrightarrow{\text{align} + \text{invert}} w_B^{(0)} \xrightarrow{\text{joint opt. with } w_A^{(0)} \sim p(z)} (w_A^*, w_B^*) \xrightarrow{\text{paste}} I.$$

3 Mathematical and Terminological Foundations

3.1 The adversarial objective

Goodfellow et al. [1] formulate generative modelling as a two-player zero-sum game. A generator $G_\theta : \mathcal{Z} \rightarrow \mathcal{X}$ maps noise $z \sim p_z$ (typically $\mathcal{N}(0, I_d)$) to samples, and a discriminator $D_\phi : \mathcal{X} \rightarrow (0, 1)$ estimates the probability that its input came from the data distribution p_{data} rather than from G . The value function is

$$V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))],$$

and training solves the minimax problem

$$\min_G \max_D V(D, G).$$

The optimal discriminator. For a fixed G inducing a distribution p_g , write the value function as an integral over $\mathcal{X}$:

$$V(D, G) = \int_{\mathcal{X}} [p_{\text{data}}(x) \log D(x) + p_g(x) \log (1 - D(x))] dx.$$

The integrand has the form $a \log t + b \log(1 - t)$, which for $a, b > 0$ is maximised on $(0, 1)$ at $t = a/(a + b)$. Hence

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

The induced generator objective. Substituting D_G^* back gives

$$C(G) = V(D_G^*, G) = \mathbb{E}_{p_{\text{data}}} \left[\log \frac{p_{\text{data}}}{p_{\text{data}} + p_g} \right] + \mathbb{E}_{p_g} \left[\log \frac{p_g}{p_{\text{data}} + p_g} \right].$$

Introducing the mixture $m = \frac{1}{2}(p_{\text{data}} + p_g)$ and adding and subtracting $\log 2$ in each expectation,

$$C(G) = -\log 4 + \text{KL}(p_{\text{data}} \| m) + \text{KL}(p_g \| m) = -\log 4 + 2 \text{JSD}(p_{\text{data}} \| p_g),$$

where JSD is the Jensen–Shannon divergence. Since $\text{JSD} \geq 0$ with equality if and only if the distributions coincide, the global minimum $C(G) = -\log 4$ is attained exactly when $p_g = p_{\text{data}}$. The adversarial game, played to optimality, is Jensen–Shannon divergence minimisation.

The non-saturating variant. In practice, minimising $\mathbb{E}_z[\log(1 - D(G(z)))]$ fails early in training. When G is poor, $D(G(z)) \approx 0$, and

$$\left. \frac{\partial}{\partial u} \log(1 - u) \right|_{u \rightarrow 0} = -1,$$

so the gradient with respect to the discriminator output stays bounded at $O(1)$ no matter how confidently the discriminator rejects the sample. Combined with the vanishing gradient that D itself contributes once it saturates, the generator receives almost no learning signal precisely when it needs it most. Goodfellow et al. therefore recommend the **non-saturating** loss: instead of minimising $\log(1 - D(G(z)))$, maximise $\log D(G(z))$, i.e. minimise

$$\mathcal{L}_G^{\text{NS}} = -\mathbb{E}_{z \sim p_z} [\log D(G(z))].$$

Here

$$\frac{\partial}{\partial u} (-\log u) = -\frac{1}{u} \xrightarrow{u \rightarrow 0} -\infty,$$

so the gradient magnitude *grows* as the discriminator becomes more confident. The two objectives share the same fixed point but have very different gradient fields away from it. StyleGAN and StyleGAN2 both use the non-saturating loss, with StyleGAN2 adding the R_1 gradient penalty [11] on the discriminator and a path-length regulariser on the generator.

3.2 Latent spaces: $\mathcal{Z}$, $\mathcal{W}$, and $\mathcal{W}+$

Understanding the distinction between these three spaces is a prerequisite for everything that follows; the entire InsetGAN optimisation lives in $\mathcal{W}+$, and the choice of space is the single most consequential decision in GAN inversion.

$\mathcal{Z}$ — **the sampling space.** $\mathcal{Z} = \mathbb{R}^{512}$ with $z \sim \mathcal{N}(0, I)$. Its distribution is fixed by fiat. This is a liability: because p_z is an isotropic Gaussian, the generator must warp it onto the data manifold, and any *imbalance* in the data (say, ninety percent of training subjects have long hair) forces the mapping to be non-uniformly stretched. Karras et al. [6] call the resulting phenomenon **entanglement**:

directions in $\mathcal{Z}$ do not correspond to single semantic factors, and interpolation paths in $\mathcal{Z}$ pass through low-density regions producing implausible images.

$\mathcal{W}$ — **the intermediate style space.** StyleGAN inserts a learned non-linear mapping network $f : \mathcal{Z} \rightarrow \mathcal{W}$, implemented as an eight-layer MLP, before the synthesis network. Crucially $\mathcal{W}$ carries *no prescribed distribution*. Because f is free to allocate volume however it likes, it can “unwarp” the sampling density: regions of $\mathcal{W}$ can be made to correspond more linearly to semantic factors. Karras et al. quantify this with perceptual path length and linear separability metrics and show $\mathcal{W}$ is measurably more disentangled than $\mathcal{Z}$.

This matters for InsetGAN in a concrete way. The joint optimisation must move w_A enough to change the body’s skin tone and shoulder geometry, without destroying the body’s realism. In an entangled space, any displacement large enough to fix skin tone would also scramble pose and garment. In $\mathcal{W}$, those factors are closer to separable, so a modest displacement can achieve a targeted change.

$\mathcal{W}+$ — **the per-layer extension.** In vanilla StyleGAN2 inference, one vector $w \in \mathbb{R}^{512}$ is broadcast to all L style inputs of the synthesis network ($L = 18$ at 1024×1024). $\mathcal{W}+$ relaxes this by allowing a *different* vector per layer:

$$\mathcal{W}+ = \underbrace{\mathcal{W} \times \dots \times \mathcal{W}}_{L \text{ times}} \cong \mathbb{R}^{L \times 512}, \quad w^+ = (w_1, \dots, w_L).$$

The dimensionality jump from 512 to $18 \times 512 = 9216$ buys enormously more expressive power, which is why essentially every inversion method [19, 20, 21, 22] targets $\mathcal{W}+$. It also buys a liability: most of $\mathcal{W}+$ does not correspond to realistic images at all, because the generator was never trained on layer-inconsistent styles. Codes far from the “diagonal” submanifold $\{(w, w, \dots, w)\}$ reconstruct the target well but *edit badly* — this is the celebrated **distortion–editability trade-off** identified by Tov et al. [21].

InsetGAN handles this with an explicit decomposition. Rather than treating the 18×512 code as free, it writes

$$w_i^+ = w^* + \delta_i, \quad i = 1, \dots, L,$$

with a single shared base code $w^* \in \mathcal{W}$ and per-layer offsets δ_i , then penalises $\sum_i \|\delta_i\|$ to keep the code near the diagonal. We return to this in Section 9.5. It is implemented in the repository verbatim: `insetgan.py` maintains `face_w_opt` (the base, shape `[1, 1, 512]`) and `face_w_delta` (the offsets, shape `[1, 18, 512]`) as two separate `requires_grad_(True)` tensors, and forms the effective code as `face_w_opt.repeat([1, 18, 1]) + face_w_delta`.

3.3 Style modulation: AdaIN and its successor

StyleGAN’s original mechanism for injecting w into the synthesis network is **adaptive instance normalisation** (AdaIN), borrowed from style transfer [10]. For a feature map x_i of channel i ,

$$\text{AdaIN}(x_i, y) = y_{s,i} \cdot \frac{x_i - \mu(x_i)}{\sigma(x_i)} + y_{b,i},$$

where $\mu(x_i)$ and $\sigma(x_i)$ are the *spatial* mean and standard deviation computed over that single feature map, and the style $y = (y_s, y_b) = A(w)$ is produced by a learned affine transform A of the latent code. The operation first destroys per-channel statistics by normalising, then rewrites them from the style. Because it is applied per-layer, styles injected at coarse resolutions (4^2 – 8^2)

control pose and global shape, mid resolutions (16^2 – 32^2) control facial features and hair style, and fine resolutions (64^2 – 1024^2) control colour scheme and micro-texture. This resolution-to-semantics correspondence is what makes **style mixing** (Section 11.3) possible: run two latents through the mapping network and use one for layers $[0, k)$ and the other for $[k, L)$.

AdaIN’s normalisation step is also, as Section 6.3 shows, the direct cause of StyleGAN’s characteristic “water droplet” artefacts, and StyleGAN2 replaces it with **weight demodulation**.

3.4 The truncation trick

Sampling from the tails of p_z produces images from low-density regions of the learned distribution, which are typically implausible. The **truncation trick** shrinks samples toward the mode. Let

$$\bar{w} = \mathbb{E}_{z \sim p_z} [f(z)]$$

be the mean latent, estimated in practice by averaging $f(z)$ over many samples (the repository uses 10,000 samples for the body generator’s `mean_latent`, and 3,000 in `insetgan.py`’s `main`). Then define

$$w' = \bar{w} + \psi(w - \bar{w}) = \psi w + (1 - \psi)\bar{w}, \quad \psi \in [0, 1].$$

At $\psi = 1$ nothing changes; at $\psi = 0$ every sample collapses to the “average” human. Intermediate values trade diversity for fidelity. This formula appears verbatim in `insetgan.py`:

```
face_mean = insgan.face_generator.mean_latent(3000)
face_w = insgan.face_generator.get_latent(torch.from_numpy(face_z).to(device))
face_w = truncation_psi * face_w + (1-truncation_psi) * face_mean
```

The InsetGAN paper additionally uses truncation as a *diversity mechanism* for initialisation: rather than always starting from $\bar{w}$, it starts from $w_{\text{trunc}} = w_{\text{rand}}(1 - \alpha) + \bar{w}\alpha$, so that different runs on the same input face yield different but still realistic bodies [15].

Truncation has a measurable cost. The InsetGAN authors report FID for their body generator rising from 13.96 untruncated to 26.67 at $\psi = 0.7$ and 71.90 at $\psi = 0.4$ — FID [12] is far more sensitive to diversity than to per-image quality, so heavier truncation looks better and scores worse.

3.5 GAN inversion as an optimisation problem

Given a frozen generator G and a target image x , inversion seeks a latent code whose decoding matches the target:

$$w^* = \arg \min_{w \in \mathcal{W}_+} \mathcal{L}(G(w), x).$$

The naive choice $\mathcal{L} = \|G(w) - x\|_2^2$ is a poor perceptual objective — it is minimised by blurry averages and is insensitive to high-frequency structure that humans care about. Practical inversion uses a composite:

$$\mathcal{L}(G(w), x) = \lambda_2 \|G(w) - x\|_2^2 + \lambda_{\text{lpips}} \mathcal{L}_{\text{LPIPS}}(G(w), x) + \lambda_{\text{id}} (1 - \langle R(G(w)), R(x) \rangle) + \lambda_{\text{reg}} \|w - \bar{w}\|_2,$$

where $\mathcal{L}_{\text{LPIPS}}$ is the learned perceptual metric of Zhang et al. [26], R is a pretrained face-recognition embedding (ArcFace [27] or similar) giving an identity cosine loss, and the final term is a prior pulling the solution toward the well-behaved centre of the latent space. This exact composition, with a $\mathcal{W}$ -norm regulariser $\|w - \bar{w}\|_2$, is implemented in the ReStyle training code shipped in this repository (`restyle/criteria/w_norm.py`).

Xia et al. [18] survey the field and classify methods into three families: **optimisation-based** (run gradient descent on w per image; most accurate, slowest), **encoder-based** (train a feed-forward network $E : \mathcal{X} \rightarrow \mathcal{W}+$; fast, less accurate), and **hybrid** (encode then refine). This repository uses an encoder (ReStyle) in the fast path and a hybrid-plus-generator-tuning method (PTI) in the high-fidelity path.

Two distinct error sources. It is worth naming them because they motivate PTI. *Distortion* is the reconstruction error $\|G(w^*) - x\|$ that remains even at the optimum, caused by x lying off the generator’s image manifold — no latent code decodes to it. *Editability* is the degree to which semantic directions still behave linearly at w^* . Pushing deeper into $\mathcal{W}+$ reduces distortion and degrades editability. PTI’s insight is that if you cannot bring the code to the image, you can bring the *image manifold to the code*, by moving the generator’s weights.

3.6 Perceptual and identity losses

Because pixel losses correlate poorly with human judgement, all four components of this system rely on deep-feature losses.

Perceptual loss [25]. Johnson, Alahi and Fei-Fei proposed comparing images through the activations of a fixed, pretrained classification network Φ rather than in pixel space:

$$\mathcal{L}_{\text{feat}}^{\Phi,j}(\hat{y}, y) = \frac{1}{C_j H_j W_j} \|\Phi_j(\hat{y}) - \Phi_j(y)\|_2^2,$$

where Φ_j is the activation of layer j . Matching deep activations enforces semantic and structural similarity while tolerating small spatial shifts that an L_2 pixel loss would punish severely.

LPIPS [26]. Zhang et al. refined this into a *calibrated* metric. Features from each layer are unit-normalised in the channel dimension, scaled by learned per-channel weights v_j , and then compared:

$$d(\hat{y}, y) = \sum_j \frac{1}{H_j W_j} \sum_{h,w} \|v_j \odot (\hat{\Phi}_j^{hw}(\hat{y}) - \hat{\Phi}_j^{hw}(y))\|_2^2,$$

where $\hat{\Phi}_j$ denotes the channel-normalised activation. The weights v_j are fit to human two-alternative-forced-choice perceptual judgements, and the paper’s central empirical claim — that deep features are an “unreasonably effective” perceptual metric across architectures and even across random initialisations — is what licences its use as a general-purpose image similarity term. Every loss term in `insetgan.py` pairs an L_1 term with an LPIPS term computed by the AlexNet-backbone variant:

```
self.lpips_loss = LPIPS(net='alex').cuda().eval()
self.l1_loss = torch.nn.L1Loss(reduction='mean')
```

Identity loss [27]. ArcFace introduced the additive angular margin softmax,

$$\mathcal{L}_{\text{ArcFace}} = -\log \frac{e^{s \cos(\theta_{y_i} + m)}}{e^{s \cos(\theta_{y_i} + m)} + \sum_{j \neq y_i} e^{s \cos \theta_j}},$$

where θ_j is the angle between the normalised feature and the normalised class centre j , m is an angular margin and s a scale. The resulting embeddings are highly discriminative on the unit hypersphere, which makes $1 - \cos(R(\hat{y}), R(y))$ an excellent identity-preservation loss. It is worth being

precise about where this does and does not appear in the present system: the **ReStyle encoder’s training objective** includes an ArcFace-lineage identity term (`restyle/criteria/id_loss.py`), so identity supervision is baked into the encoder’s weights, but the **InsetGAN joint optimisation itself uses only $L_1 + \text{LPIPS}$** — no identity network is evaluated during the optimisation loop. Identity is preserved indirectly, via the face reconstruction loss that anchors the optimised face to the reference face produced from the inverted code. This is a real architectural limitation and is revisited in Section 12.

4 Classical and Foundational Generative Approaches

This section is deliberately brief; its purpose is to establish why the StyleGAN architecture, and not some alternative, is the substrate on which InsetGAN could be built.

4.1 Variational autoencoders

Kingma and Welling [2] approach generation through explicit latent-variable modelling. Assume data are generated by sampling $z \sim p(z)$ and then $x \sim p_\theta(x | z)$. Exact maximum likelihood is intractable because $p_\theta(x) = \int p_\theta(x | z)p(z) dz$ has no closed form, so a variational posterior $q_\phi(z | x)$ is introduced and the evidence lower bound is optimised:

$$\log p_\theta(x) \geq \mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)).$$

The reparameterisation trick, $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, makes the expectation differentiable with respect to ϕ .

VAEs have two properties that are attractive for the present application and one that is disqualifying. Attractive: they come with an *encoder for free* — inversion is not a separate research problem — and training is stable. Disqualifying: the reconstruction term is typically a per-pixel Gaussian likelihood, i.e. an L_2 loss, and averaging over the posterior systematically produces blurry samples. At 1024×1024 , VAE outputs in 2022 were not remotely competitive with GANs on photorealism. The practical compromise the field adopted — and that this project embodies — is to use a GAN for synthesis and to solve inversion separately with a *learned encoder* trained against the frozen GAN, which is exactly what pSp and ReStyle are.

4.2 Early GANs and their failure modes

The original GAN formulation with convolutional generators suffered from three well-catalogued problems: **mode collapse** (the generator maps many z to a narrow set of outputs, satisfying the discriminator while ignoring most of p_{data}), **training instability** (the minimax game has no descent-direction guarantee; the discriminator can overpower the generator and kill its gradients, or oscillate), and **resolution ceilings** (naive scaling to high resolution makes the discriminator’s task trivially easy, since real and fake high-frequency statistics differ sharply, which again starves the generator).

Radford, Metz and Chintala’s DCGAN [3] provided the first architectural recipe that trained reliably: strided and fractionally-strided convolutions instead of pooling, batch normalisation in both networks, ReLU in the generator with tanh output, LeakyReLU in the discriminator, and no fully-connected hidden layers. DCGAN also demonstrated the first convincing evidence of *semantic*

latent arithmetic — the “man with glasses – man + woman = woman with glasses” result — establishing that GAN latent spaces encode interpretable structure. That observation is the direct intellectual ancestor of the latent-direction editing used in Section 11.2.

Subsequent work attacked stability from the loss side. Wasserstein GAN [4] replaced the Jensen–Shannon objective with the Earth-Mover distance under a Lipschitz constraint, giving a critic whose loss correlates with sample quality and does not saturate. Mescheder et al. [11] analysed local convergence and introduced the R_1 gradient penalty on real data,

$$R_1(\phi) = \frac{\gamma}{2} \mathbb{E}_{x \sim p_{\text{data}}} [\|\nabla_x D_\phi(x)\|^2],$$

which is the regulariser StyleGAN2 actually uses — the InsetGAN authors report sweeping γ between 0.1 and 20 and settling on $\gamma = 13$ for their body generator [15].

4.3 Why StyleGAN was a discontinuity

Two ideas, neither of which is a loss-function change, account for most of the jump.

Progressive growing [5] trained generator and discriminator starting at 4×4 and incrementally faded in higher-resolution layers. This turned a single hard optimisation into a curriculum of easy ones: the network first learns global structure at low resolution, where the discriminator cannot exploit high-frequency statistics, and only then learns detail. It made 1024×1024 synthesis feasible for the first time and produced the CelebA-HQ results that defined the state of the art in 2018.

Style-based synthesis [6] restructured *where the latent code enters*. In every prior architecture, z was fed to the first layer and propagated forward. StyleGAN instead starts the synthesis network from a learned constant tensor and injects $w = f(z)$ at every layer through AdaIN. This is a strictly more expressive interface — the latent influences all scales directly rather than through a bottleneck — and it produces the scale-separated semantics (coarse = pose, middle = features, fine = colour) that all downstream editing work depends on. It is not an exaggeration to say that InsetGAN, ReStyle, PTI, InterFaceGAN, StyleSpace and SeFa are all *consequences* of that one architectural decision.

5 Research Evolution: From DCGAN to Multi-GAN Compositing

This section traces the lineage the project stands on, emphasising the specific problem each step solved.

5.1 Stage 1: architectural stabilisation (2015–2018)

DCGAN [3] established a trainable convolutional recipe. Progressive growing [5] broke the resolution ceiling by curriculum. By 2018, 1024^2 unconditional face synthesis was possible but the latent space was entangled and the images, while sharp, lacked the fine stochastic detail (individual hairs, pores, freckles) that reads as photographic.

5.2 Stage 2: style-based generation (2019–2020)

StyleGAN [6] introduced the mapping network, AdaIN style injection, per-layer noise inputs for stochastic detail, mixing regularisation, and the truncation trick. It also introduced two characteristic pathologies: blob-shaped “water droplet” artefacts present in nearly every generated image,

and *phase artefacts* where features such as teeth or eyes stick to fixed pixel locations rather than following the head pose.

StyleGAN2 [7] diagnosed both. The droplets were traced to AdaIN’s per-map normalisation and fixed by replacing normalisation with **weight demodulation** (derived in Section 6.3). The phase artefacts were traced to progressive growing itself and fixed by replacing it with a skip-connection generator and a residual discriminator trained at full resolution throughout. StyleGAN2 also added **path length regularisation**, which encourages the generator’s Jacobian to have uniform singular values,

$$\mathcal{L}_{\text{path}} = \mathbb{E}_{w, y \sim \mathcal{N}(0, I)} \left[(\|\mathbf{J}_w^\top y\|_2 - a)^2 \right], \quad \mathbf{J}_w = \frac{\partial G(w)}{\partial w},$$

with a an exponential moving average of the observed norms. The practical consequence is that fixed-size steps in $\mathcal{W}$ produce roughly fixed-magnitude image changes — a *smoother, better-conditioned* latent space, which is precisely what makes gradient-based latent optimisation (InsetGAN, PTI) well-behaved.

StyleGAN2-ADA [8] then made high-quality training feasible on limited data through adaptive discriminator augmentation, applying differentiable augmentations to discriminator inputs with a probability tuned online by an overfitting heuristic. Both StyleGAN-Human and the InsetGAN body generator were trained with the ADA architecture. StyleGAN3 [9] later addressed aliasing and texture sticking; the StyleGAN-Human release ships a StyleGAN3 variant, and `generate.py` in this repository accepts `--version 3`, although the InsetGAN path uses the StyleGAN2 checkpoint.

5.3 Stage 3: domain specialisation (2022)

StyleGAN-Human [13] is a *data-centric* study rather than an architectural one. Fu et al. collected and annotated a large-scale human image dataset of over 230{,}000 samples “capturing diverse poses and textures,” and systematically studied three axes of data engineering: data size, data distribution, and data alignment. Their central finding is that alignment matters enormously — humans must be normalised into a canonical frame before training, exactly as FFHQ normalises faces on eye positions — and that a StyleGAN2 trained on properly curated and aligned human data at portrait aspect produces the best full-body results then available. They released checkpoints for StyleGAN1, StyleGAN2 and StyleGAN3 variants, along with editing, style-mixing, interpolation and InsetGAN implementations. The checkpoint this repository downloads, `stylegan2_1024.pkl` (362 MB), is the StyleGAN2 1024 portrait model from that release.

The InsetGAN paper’s own body generator is a parallel effort: 83{,}972 proprietary photographs, pose-normalised on a neck-to-hip upper-body axis using a pose-detection network [40], backgrounds enlarged by reflection padding and heavily blurred with a 27-pixel Gaussian kernel to force generator capacity onto the foreground, trained for 28 days and 18 hours on four Titan V GPUs at batch size 4 for 42M images shown [15]. They also report a DeepFashion [14] variant: 10{,}145 images at 1024×768 , 9 days on four V100s, 18M images.

5.4 Stage 4: the inversion problem and encoder-based solutions

Once a strong frozen generator exists, the bottleneck moves to *getting real images into it*.

Optimisation-based inversion. Abdal et al. [19] showed that direct gradient descent on $w^+ \in \mathcal{W}^+$ with a VGG perceptual loss can embed essentially any image — including images far outside

the training domain, such as cats and paintings, into a face generator. The cost is minutes per image and, as they note, codes that reconstruct well but edit poorly.

pSp [20]. Richardson et al. built the first strong feed-forward encoder. A ResNet-IR backbone with a feature pyramid produces L style vectors via small `map2style` heads — coarse styles from the deepest feature level, medium and fine from shallower levels — and each is added to $\bar{w}$. Training uses L_2 , LPIPS, an ArcFace identity loss, and a $\mathcal{W}$ -norm regulariser. pSp reframed inversion as *image-to-image translation through a frozen StyleGAN*, so the same architecture handles sketch-to-face, segmentation-to-face and super-resolution.

e4e [21]. Tov et al. made the distortion–editability trade-off explicit. They observe that codes far from the $\mathcal{W}$ diagonal reconstruct better but edit worse, and design an encoder that deliberately predicts a *small deviation* from a single base code, with a progressive training schedule and an adversarial latent discriminator that pushes predicted codes toward the real $\mathcal{W}$ distribution. This repository ships both encoders (`restyle/models/psp.py`, `restyle/models/e4e.py`); the notebooks download both checkpoints and select pSp, with e4e commented out and annotated as “better editability at a slight quality cost.”

ReStyle [22]. Alaluf, Patashnik and Cohen-Or observed that a single forward pass is an unnecessarily hard constraint, and replaced it with iterative residual refinement. This is the encoder actually used in the production path of this repository; Section 7 derives it.

5.5 Stage 5: moving the generator instead of the code

PTI [23]. Roich et al. broke the distortion–editability trade-off from the other side. Invert into the *well-behaved* native $\mathcal{W}$ space, accepting some distortion, then fine-tune the generator’s weights so that this “pivot” code decodes to the target exactly, with a locality regulariser that confines the weight change to a neighbourhood of the pivot. Section 8 derives it. This repository implements PTI as an alternative front-end in `server/User_Whole_Body_Generation_Using_PTI.ipynb`.

A related idea, PULSE [24], searched the latent space under a downsampling consistency constraint for super-resolution; the InsetGAN authors explicitly draw the analogy, noting that applying losses at low resolution “allows for more flexibility during optimization” in a strategy “similar to PULSE” [15].

5.6 Stage 6: compositing multiple specialist GANs

The final step is the one this document is about. Prior attempts to raise effective resolution by combining generators were *tiling*-based: Frühstück et al.’s earlier TileGAN [16] synthesised very large textures by sequentially producing tiles. As the InsetGAN paper argues, tiling fails when the coupling between parts is non-local — a face and a body must agree on skin tone and gender, which is not a tile-boundary property.

The alternative baseline is *inpainting*: mask the face region and fill it with a conditional model such as CoModGAN [17]. Frühstück et al. trained CoModGAN with square holes around faces for two weeks on four V100s and report that InsetGAN produces “more convincing results”; in their user study, 98% of participants preferred InsetGAN’s joint-optimisation output over the unrefined body-GAN image, whereas only 7% preferred CoModGAN’s [15].

InsetGAN’s framing is different from both: no tiling, no inpainting, no new training. Two frozen generators, one shared objective, and gradient descent in two latent spaces simultaneously.

6 StyleGAN2 in Detail

Both generators in this system are StyleGAN2. Understanding their internals is necessary to understand what the InsetGAN optimisation is actually manipulating.

6.1 The mapping network

$$f: \mathcal{Z} \rightarrow \mathcal{W}, \quad w = f(z), \quad z \in \mathbb{R}^{512}, w \in \mathbb{R}^{512}.$$

f is an eight-layer MLP with 512 units per layer and leaky-ReLU activation (slope 0.2); the input z is first normalised to the unit hypersphere. The repository’s generator config makes this explicit:

```
config = {"latent": 512, "n_mlp": 8, "channel_multiplier": 2}
self.body_generator = bodyGAN(size=1024, style_dim=config["latent"],
                               n_mlp=config["n_mlp"],
                               channel_multiplier=config["channel_multiplier"])
```

The output w is then transformed by a *separate* learned affine map A_ℓ per synthesis layer ℓ , producing the per-layer style vector $s_\ell = A_\ell(w) \in \mathbb{R}^{C_\ell^{\text{in}}}$, where C_ℓ^{in} is that layer’s input channel count.

Two facts about f matter downstream. It has no prescribed output distribution, which is what permits disentanglement. And it is *not used* in the InsetGAN optimisation: the optimisation variables are w -space codes, so f appears only when sampling an initial code and when computing $\bar{w}$.

6.2 The synthesis network

Synthesis begins from a learned constant $x_0 \in \mathbb{R}^{512 \times 4 \times 4}$ and proceeds through $\log_2(1024) - 1 = 9$ resolution blocks, each containing two modulated convolutions (the first with $2\times$ upsampling) plus a toRGB branch. At 1024×1024 there are

$$L = 2 \times 9 = 18$$

style inputs, which is exactly the 18 appearing everywhere in the InsetGAN code (`face_w_opt.repeat([1, 18, 1])`).

Each block also receives a per-pixel noise input scaled by a learned scalar, $x \leftarrow x + b \cdot n$ with $n \sim \mathcal{N}(0, I)$ spatially i.i.d. This decouples *stochastic* detail (exact hair placement, pore distribution) from *semantic* content (which is carried by w). During inversion and joint optimisation, noise is held fixed — every generator call in `insetgan.py` passes `randomize_noise=False` — because randomised noise would inject non-determinism into the loss and make gradient descent chase a moving target.

StyleGAN2 replaced progressive growing with a **skip generator**: each resolution block emits an RGB image which is upsampled and summed into a running total, so all resolutions contribute to the output simultaneously and the network can shift its effective depth during training without a discrete curriculum. The discriminator uses residual connections.

6.3 Weight demodulation: derivation

This is the central architectural change from StyleGAN to StyleGAN2, and it is worth deriving because the mechanism reappears verbatim in the repository’s `edit/edit_helper.py`.

The problem. AdaIN normalises each feature map to zero mean and unit variance and then rewrites its statistics from the style. Karras et al. [7] observed that this gives the generator a perverse escape hatch: if it needs to sneak signal magnitude past the normalisation, it can create a strong, spatially localised spike in one feature map. Because instance normalisation divides by the map’s standard deviation, and the spike dominates that standard deviation, the *rest* of the map is scaled down — effectively letting the generator control global magnitude through a local artefact. The droplet is that spike, and it is visible in nearly every StyleGAN-1 image once one knows to look.

The fix. Note that modulation followed by convolution is a *linear* operation on the weights, so both can be folded into the weights and the normalisation can be applied *statistically* rather than to actual data.

Let the layer weight be w_{ijk} with i indexing input channels, j output channels and k the spatial kernel position. Style modulation scales the input channels:

$$w'_{ijk} = s_i \cdot w_{ijk},$$

where s_i is the i -th component of the style vector. This is exactly equivalent to scaling the input feature maps before the convolution, which is what AdaIN’s second step does.

Now consider what the subsequent normalisation is *for*: restoring unit variance to the output. Assume the input activations are i.i.d. with unit variance — the assumption Karras et al. make explicitly. Then the variance of output channel j is

$$\sigma_j^2 = \sum_{i,k} (w'_{ijk})^2,$$

since a convolution is a sum of products of independent unit-variance terms weighted by w' . To restore unit output variance we simply divide by σ_j , giving the **demodulated** weights

$$w''_{ijk} = \frac{w'_{ijk}}{\sqrt{\sum_{i,k} (w'_{ijk})^2 + \epsilon}}$$

with ϵ a small constant for numerical stability.

Why this removes the droplets. The demodulation is computed from the *weights*, not from the data. It is therefore a deterministic per-layer rescaling that cannot be manipulated by planting a spike in an activation map. The generator loses the escape hatch, and the artefact disappears. The normalisation is now based on a statistical *expectation* of the signal rather than its measured realisation — weaker in principle, sufficient in practice, and cheaper (it is one extra reduction over the weight tensor, and it can be folded into a grouped convolution).

The repository contains a faithful reimplementation of exactly this arithmetic, used by the attribute-editing code to intercept and modify per-layer styles (StyleGAN-Human/edit/edit_helper.py):

```
def conv_warper(layer, input, style, noise):
    conv = layer.conv
    batch, in_channel, height, width = input.shape
    style = style.view(batch, 1, in_channel, 1, 1)
    weight = conv.scale * conv.weight * style           # modulate: w' = s_i * w
```

```

if conv.demodulate:
    demod = torch.rsqrt(weight.pow(2).sum([2, 3, 4]) + 1e-8)
    weight = weight * demod.view(batch, conv.out_channel, 1, 1, 1)  #  $w'' = w' / \sigma_j$ 

```

The `sum([2, 3, 4])` reduces over input channels and both kernel spatial dimensions — precisely the $\sum_{i,k}$ of the derivation — and `rsqrt` gives $1/\sigma_j$. Note also the grouped-convolution trick that follows: the per-sample modulated weights are reshaped to `(batch * out_channel, in_channel, k, k)` and the batch is folded into the channel dimension with `groups=batch`, so a single `F.conv2d` call applies a different modulated kernel to each sample.

6.4 The truncation trick in the synthesis network

Applying truncation in $\mathcal{W}$ rather than $\mathcal{Z}$ is important: $\bar{w} = \mathbb{E}_z[f(z)]$ is the centroid of the *learned* style distribution, and moving toward it moves toward high-density, high-quality regions. StyleGAN implementations typically allow truncation to be applied only to the coarse layers (preserving fine detail diversity), but the code in this repository applies it uniformly:

```
w = G.mapping(z, label, truncation_psi=truncation_psi)
```

in `generate.py`, and the explicit interpolation form in `insetgan.py` shown in Section 3.4.

6.5 Style mixing and mixing regularisation

During training, StyleGAN uses **mixing regularisation**: with some probability, two codes $w_1 = f(z_1)$ and $w_2 = f(z_2)$ are sampled and a random crossover point k chosen, so that layers $[0, k)$ receive w_1 and $[k, L)$ receive w_2 . This prevents the network from assuming correlation between adjacent layers’ styles and directly enforces the scale-separation that makes inference-time style mixing work.

At inference, the same mechanism becomes a creative control. The repository’s `style_mixing.py` builds a grid where each cell composites a row seed’s styles into a column seed’s image over a chosen layer range:

```

python style_mixing.py --outdir=outputs/stylemixing \
    --rows=85,100,75,458,1500,86 --cols=55,821,1789,293,75 \
    --network=pretrained_models/stylegan2_1024.pkl --styles=0-3

```

`--styles=0-3` selects the four coarsest style layers, which for a full-body model transfers pose and global body shape while retaining the column image’s clothing colours and texture.

7 The ReStyle Encoder: Iterative Residual Inversion

7.1 The limitation of single-pass encoders

A conventional encoder E must, in one forward pass, map an arbitrary image x to a point $w \in \mathcal{W} +$ such that $G(w) \approx x$. This is a hard regression problem: the encoder never observes the consequence of its own prediction, so it must learn the inverse of a highly nonlinear 1024^2 -output function open-loop. Alaluf et al. [22] characterise this as an unnecessarily strong constraint on training, and relax it.

7.2 The iterative scheme

ReStyle introduces a *closed loop*. Let E be the encoder and G the frozen generator. The process is initialised with the average latent code and its decoding,

$$w_0 = \bar{w}, \quad \hat{y}_0 = G(w_0).$$

At step t , the encoder is given a six-channel input formed by concatenating the target with the current reconstruction along the channel axis,

$$x_t := x \parallel \hat{y}_t \in \mathbb{R}^{6 \times H \times W},$$

and predicts a **residual** in latent space,

$$\Delta_t := E(x_t),$$

which updates the code additively,

$$w_{t+1} \leftarrow \Delta_t + w_t,$$

and the reconstruction is refreshed,

$$\hat{y}_{t+1} := G(w_{t+1}).$$

These are Equations (1)–(4) of the ReStyle paper [22]. The recursion is run for N steps; the conventional single-pass encoder is the special case $N = 1$.

Written as a single recurrence, the scheme is

$$w_{t+1} = w_t + E(x \parallel G(w_t)),$$

which is a fixed-point iteration on w . A fixed point satisfies $E(x \parallel G(w^*)) = 0$, i.e. the encoder reports “no correction needed.”

7.3 Why iteration helps

Three arguments, all of which the paper supports empirically.

Residuals are easier to learn than absolutes. At step t the encoder sees both the target and its current best guess. The quantity it must produce is a *correction*, whose magnitude shrinks as the reconstruction improves. Alaluf et al. observe exactly this decay: “the amount of change decreases with each step.” Learning a small correction conditioned on the current error is a better-conditioned regression than learning the full code from scratch.

Error feedback. The encoder is explicitly informed of its own mistake. Regions where $\hat{y}_t$ differs from x are visible in the concatenated input, so attention can be allocated where it is needed. The paper’s analysis of which regions change at each step shows the model first fixes global attributes (pose, colour) and then progressively refines localised detail (hair strands, background, accessories).

Relaxed architectural demands. Because the multi-step process supplies capacity through iteration rather than depth, ReStyle can use a *simpler* encoder than pSp: all k style vectors are extracted from a single final 16×16 feature map through `map2style` blocks, rather than from three hierarchical feature levels. Iteration substitutes for architectural complexity.

The cost is N forward passes through E and G instead of one. Alaluf et al. report convergence with $N < 10$ and train with $N = 5$. This is still orders of magnitude faster than per-image optimisation: ReStyle inversion in this project takes roughly ten seconds, against approximately two minutes for PTI.

7.4 Training objective

The encoder is trained with the losses computed *at every step* — so back-propagation occurs N times per batch — using the pSp-lineage composite:

$$\mathcal{L}(x, \hat{y}_t) = \lambda_2 \|x - \hat{y}_t\|_2 + \lambda_{\text{lips}} \mathcal{L}_{\text{LPIPS}}(x, \hat{y}_t) + \lambda_{\text{id}} (1 - \langle R(x), R(\hat{y}_t) \rangle) + \lambda_w \mathcal{L}_{\mathcal{W}}(w_t),$$

with the latent regulariser implemented in this repository as (`restyle/criteria/w_norm.py`):

```
def forward(self, latent, latent_avg=None):
    if self.start_from_latent_avg:
        latent = latent - latent_avg
    return torch.sum(latent.norm(2, dim=(1, 2))) / latent.shape[0]
```

i.e. $\mathcal{L}_{\mathcal{W}}(w) = \|w - \bar{w}\|_2$ averaged over the batch. The identity term uses a pretrained face-recognition backbone (`restyle/criteria/id_loss.py`), which is where ArcFace-lineage supervision [27] enters this system.

7.5 The implementation in this repository

The recursion is implemented in `restyle/utils/inference_utils.py` and is worth quoting in full because it maps one-to-one onto the equations above:

```
def run_on_batch(inputs, net, opts, avg_image):
    y_hat, latent = None, None
    results_batch = {idx: [] for idx in range(inputs.shape[0])}
    results_latent = {idx: [] for idx in range(inputs.shape[0])}
    for iter in range(opts.n_iters_per_batch):
        if iter == 0:
            avg_image_for_batch = avg_image.unsqueeze(0).repeat(inputs.shape[0], 1, 1, 1)
            x_input = torch.cat([inputs, avg_image_for_batch], dim=1)
        else:
            x_input = torch.cat([inputs, y_hat], dim=1)

        y_hat, latent = net.forward(x_input,
                                   latent=latent,
                                   randomize_noise=False,
                                   return_latents=True,
                                   resize=opts.resize_outputs)
        ...
    y_hat = net.face_pool(y_hat)
    return results_batch, results_latent
```

`torch.cat([inputs, y_hat], dim=1)` is Equation (1); the `latent=latent` argument carries w_t into the network, where `models/psp.py` performs the additive update

```
if x.shape[1] == 6 and latent is not None:
    codes = codes + latent          # w_{t+1} = Delta_t + w_t
else:
    codes = codes + self.latent_avg.repeat(codes.shape[0], 1, 1)  # w_1 = Delta_0 + w_avg
```

which is Equation (3), with the $t = 0$ branch supplying the $\bar{w}$ initialisation. `net.face_pool(y_hat)` downsamples the reconstruction back to the encoder’s input resolution before the next iteration.

The notebooks configure five steps, matching the paper’s training setting:

```
opts.n_iters_per_batch = 5
opts.resize_outputs = False
```

and `run_on_batch` returns *all* intermediate latents, one per step. This detail matters for Section 10: the pipeline selects the final refinement with `face_latent_codes[0][4]` — index 4 of 5, i.e. w_5 , the last iterate.

7.6 A note on the portrait-aspect decoder

A subtle engineering issue in this repository deserves recording, because it illustrates a real property of the architecture. The ReStyle pSp checkpoint (`restyle_psp_ffhq_encode.pt`) was trained against a *square* 1024×1024 FFHQ decoder, whereas the whole-body decoder is *portrait* 1024×512 . Loading the checkpoint’s decoder weights into the portrait model fails, because the per-layer noise buffers have different spatial shapes. The notebooks resolve this by monkey-patching `pSp.load_weights` to filter the decoder state dict by shape:

```
decoder_sd = self._pSp__get_keys(ckpt, 'decoder')
model_sd    = self.decoder.state_dict()
filtered    = {k: v for k, v in decoder_sd.items()
               if k in model_sd and v.shape == model_sd[k].shape}
```

The notebook’s own annotation explains why this is correct rather than merely expedient: the eighteen skipped tensors (`input.input`, `noises.noise_0` through `noises.noise_16`) “are not trained parameters; they are per-resolution noise maps whose shapes are determined by the generator architecture, not the training data.” The convolutional weights — which *are* the learned model — all load. This is a direct practical consequence of StyleGAN2’s separation of semantic content (w , carried by weights) from stochastic detail (noise, carried by buffers), discussed in Section 6.2.

8 Pivotal Tuning Inversion

8.1 Motivation

ReStyle produces a code in $\mathcal{W}+$ quickly, but for an out-of-domain face — unusual lighting, heavy makeup, headwear, an identity poorly represented in FFHQ — residual distortion remains. Pushing further into $\mathcal{W}+$ reduces distortion but degrades editability. Roich et al. [23] observe that this trade-off is only forced if the generator is treated as immovable, and propose moving it.

The governing intuition, in their words, is that because of StyleGAN’s disentangled structure, “slight and local changes to its produced appearance can be applied without damaging its powerful editing capabilities.” So: find the closest *editable* point to the target, then deform the generator locally so that this point decodes to the target exactly.

8.2 Stage 1: inversion to the pivot

PTI inverts into the **native** $\mathcal{W}$ space, not $\mathcal{W}+$ — a deliberate choice, since $\mathcal{W}$ has the best editability and the distortion will be eliminated in stage 2 anyway. The generator is frozen. The

objective, Equation (1) of the paper, jointly optimises the latent and the noise maps:

$$w_p, n = \arg \min_{w, n} \mathcal{L}_{\text{LPIPS}}(x, G(w, n; \theta)) + \lambda_n \mathcal{L}_n(n),$$

where $\mathcal{L}_n$ is the StyleGAN2 projector’s noise regulariser, which penalises spatial autocorrelation in the noise maps to prevent them from absorbing signal that belongs in w . Note the mapping network is bypassed entirely: the optimisation variable is w directly, not z . In the paper’s words, “we do not use StyleGAN’s mapping network (converting from Z to W).”

The result w_p is the **pivot**.

8.3 Stage 2: pivotal tuning

Now freeze w_p and unfreeze the generator. Let $x_p = G(w_p; \theta^*)$ be the image produced by the pivot under the *tuned* weights. The fine-tuning loss (Equation 2 of the paper) is

$$\mathcal{L}_{pt} = \mathcal{L}_{\text{LPIPS}}(x, x_p) + \lambda_{L2} \mathcal{L}_{L2}(x, x_p),$$

with the generator initialised from the pretrained θ . For N images simultaneously (multi-identity personalisation), this extends to

$$\mathcal{L}_{pt} = \frac{1}{N} \sum_{i=1}^N \left(\mathcal{L}_{\text{LPIPS}}(x_i, x_{p_i}) + \lambda_{L2} \mathcal{L}_{L2}(x_i, x_{p_i}) \right), \quad x_{p_i} = G(w_i; \theta^*).$$

Roich et al. emphasise that using the *pivot* is essential: initialising from random or mean codes “lead to unsuccessful convergence.”

8.4 Stage 3: locality regularisation

Tuning the generator to reproduce one image damages it elsewhere — Roich et al. call this a “ripple effect,” and it is severe when tuning for several identities. The regulariser confines the damage.

In each iteration, sample $z \sim \mathcal{N}(0, I)$, map it through the (frozen) mapping network to $w_z = f(z)$, and construct an interpolated code at a fixed distance α from the pivot along the direction of w_z (Equation 4):

$$w_r = w_p + \alpha \frac{w_z - w_p}{\|w_z - w_p\|_2}.$$

Then require that the tuned generator agree with the *original* generator at w_r . With $x_r = G(w_r; \theta)$ from the original weights and $x_r^* = G(w_r; \theta^*)$ from the tuned weights (Equation 5):

$$\mathcal{L}_R = \mathcal{L}_{\text{LPIPS}}(x_r, x_r^*) + \lambda_{L2}^R \mathcal{L}_{L2}(x_r, x_r^*),$$

extended over N_r samples per iteration (Equation 6):

$$\mathcal{L}_R = \frac{1}{N_r} \sum_{i=1}^{N_r} \left(\mathcal{L}_{\text{LPIPS}}(x_{r,i}, x_{r,i}^*) + \lambda_{L2}^R \mathcal{L}_{L2}(x_{r,i}, x_{r,i}^*) \right).$$

The complete second-stage optimisation (Equation 7) is

$$\theta^* = \arg \min_{\theta^*} \mathcal{L}_{pt} + \lambda_R \mathcal{L}_R.$$

Reading the regulariser. $\mathcal{L}_R$ is a *self-distillation* term: the pretrained generator is its own teacher everywhere except near the pivot. The normalisation $\|w_z - w_p\|_2$ places every sampled anchor at exactly distance α from the pivot, so the regulariser defines a sphere of radius α around w_p on which the generator must not change. The published hyperparameters are $\lambda_{L2} = 1$, $\lambda_{\text{LPIPS}} = 1$, $\alpha = 30$, $\lambda_{L2}^R = 1$, $\lambda_R = 0.1$, $N_r = 1$. Roich et al. report that inversion takes approximately one minute, and tuning under a minute without regularisation, under two with — growing linearly with the number of identities.

8.5 PTI in this repository

server/User_Whole_Body_Generation_Using_PTI.ipynb clones <https://github.com/danielroich/PTI> and configures it against the same checkpoints used elsewhere:

```
paths_config.stylegan2_ada_ffhq = f'{PTI_base}/pretrained_models/ffhq.pkl'
paths_config.dlib = f'{Style_human_base}/pretrained_models/shape_predictor_68_face_landmarks.dlib'
hyperparameters.use_locality_regularization = False
```

Two decisions are visible here. First, the locality regulariser is **disabled** — a defensible choice in this application, because the tuned generator is used for exactly one purpose (decoding this one identity inside the InsetGAN loop) and is never asked to perform semantic edits elsewhere in the latent space, so the ripple effect costs nothing and the regulariser’s extra forward passes are pure overhead. Second, the dlib [39] landmark predictor is shared with the StyleGAN-Human alignment path, keeping the face crop convention consistent across the two front-ends.

The wrapper that produces latents is:

```
def get_latent_code_using_PTI(path_to_original_image_folder):
    os.chdir(PTI_base)
    pre_process_images(f'{PTI_base}/image_original')
    model_id = run_PTI(use_wandb=False, use_multi_id_training=use_multi_id_training)
    ...
    w_pivot = torch.load(f'{embedding_dir}/0.pt')      # the pivot code w_p
    np.savez(f'w_latents.npz', w=w_pivot.cpu().detach().numpy())
```

The notebook’s own header records the trade-off honestly: “PTI fine-tunes the generator weights per image for higher-fidelity face reconstruction, at the cost of longer inversion time (~2 minutes vs ~10 seconds for ReStyle pSp).”

An important subtlety: PTI returns a *tuned generator* θ^* as well as a pivot code. The InsetGAN path in this repository consumes only the latent code and runs it through the *original* FFHQ generator. This means the PTI-specific gain — the part of the reconstruction fidelity that lives in the tuned weights rather than in w_p — is partly discarded. Fully exploiting PTI here would require loading θ^* into `InsetGAN.face_generator`. This is noted in Section 12 as an implementation limitation.

9 InsetGAN: The Joint Optimisation Objective

This is the core of the system. All notation follows the primary source [15].

9.1 Setup and notation

- G_A — the **canvas** generator; here, StyleGAN-Human at 1024. Produces $I_A := G_A(w_A)$.
- G_B — the **inset** generator; here, StyleGAN2-FFHQ at 1024. Produces $I_B := G_B(w_B)$.
- $\mathcal{B}(\cdot)$ — bounding-box crop of the canvas image at the face region, located by a face detector. $\mathcal{B}(I_A)$ is the face region *as synthesised by the body GAN*.
- $D_{64}(\cdot)$ — downsampling to 64×64 (the paper specifies only the target resolution; the reference implementation uses area interpolation).
- $E_x(\cdot)$ — the border region of an image, a frame of width x pixels around its perimeter.
- $R^I(\cdot)$, $R^O(\cdot)$ — the interior region of the face crop, and the body region *outside* the face bounding box, respectively.
- I_{ref} — a reference image held fixed during optimisation (a reference body for $\mathcal{L}_{RB}$, a reference face for $\mathcal{L}_{RF}$).

Both generators are frozen: `.eval().requires_grad_(False).cuda()` in the repository’s constructor. The optimisation variables are w_A and w_B only.

9.2 The master problem

Equation (1) of the paper:

$$\min_{w_A, w_B} \int_{\Omega} \mathcal{L}(G_A(w_A), G_B(w_B)), \quad \Omega := \mathcal{B}(G_A(w_A)).$$

As the authors write, $\mathcal{L}$ “captures the loss both along the boundary of Ω measuring seam quality and inside the region Ω measuring similarity of I_A and I_B inside the respective faces,” and “the full optimization is complex as the region of interest Ω depends on w_A .”

The paper decomposes $\mathcal{L}$ into three objectives:

1. **Coarse appearance agreement.** The face regions produced by the two generators must match at a coarse scale, so that attributes such as skin tone are consistent between the pasted face and the surrounding neck.
2. **Boundary agreement.** The pixels immediately around the crop boundary must match so that a plain copy-and-paste leaves no seam.
3. **Realism.** The composed result must remain on the natural image manifold.

9.3 Loss term 1: coarse appearance, $\mathcal{L}_A$

Downsample both face regions to 64×64 and compare:

$$I_A^\downarrow = D_{64}(\mathcal{B}(I_A)), \quad I_B^\downarrow = D_{64}(I_B),$$

$$\mathcal{L}_A := \lambda_1 \mathcal{L}_1(I_A^\downarrow, I_B^\downarrow) + \lambda_2 \mathcal{L}_{\text{LIPS}}(I_A^\downarrow, I_B^\downarrow)$$

(Equation 2).

Why downsample to 64×64 ? Two reasons, both stated by the authors. It “allow[s] for more flexibility during optimization” — matching at low resolution constrains only the low-frequency content (skin tone, illumination, head pose, hair mass) and leaves the face generator free to supply its own high-frequency identity detail, which is the entire point of using a specialist. And it “reduce[s] the risk of overfitting to artifacts from the source image” — the body GAN’s face region

is precisely the low-quality region we are trying to replace, so matching it at full resolution would import its defects.

Why L_1 rather than L_2 ? The paper states directly: “we observe a slightly better visual performance when using L_1 over L_2 losses.” L_1 is more robust to outliers, so a small region of large disagreement (a stray hair strand, a specular highlight) does not dominate the gradient.

9.4 Loss term 2: boundary blending, $\mathcal{L}_B$

This is the term that makes the composite seamless. Take the border frame of width 8 pixels of both the canvas crop and the inset image, at **full resolution**, and match them:

$$\mathcal{L}_B := \lambda_3 \mathcal{L}_1(E_8(\mathcal{B}(I_A)), E_8(I_B)) + \lambda_4 \mathcal{L}_{\text{LPIPS}}(E_8(\mathcal{B}(I_A)), E_8(I_B))$$

(Equation 3), “where $E_x(I)$ is the border region of I of width x pixels.”

The design is worth dwelling on. The composite operation is a hard rectangular replacement — no alpha feathering, no Poisson blending, no learned harmoniser. A seam is visible precisely when the pixel values on either side of the rectangle boundary disagree. By forcing the two generators to agree on an 8-pixel-wide frame at full resolution, the optimisation makes the boundary *locally continuous by construction*. Everything inside the frame is then free: the inset generator supplies the face, the canvas generator supplies the body, and neither needs to match the other except in that thin annulus.

This is a genuinely elegant reduction. Image compositing is normally treated as a blending problem in *pixel* space; InsetGAN moves it to a constraint-satisfaction problem in *latent* space, where the generators’ own priors guarantee that any solution is a realistic image.

Note also the asymmetry in weights between $\mathcal{L}_A$ and $\mathcal{L}_B$ in the reference implementation. Coarse matching uses $\lambda_1 = 500$; boundary matching uses λ_3 up to 25,000 in the joint face step — fifty times larger. The seam is the hard constraint; coarse appearance is a soft preference.

9.5 Loss term 3: latent regularisation, $\mathcal{L}_R$

$$\mathcal{L}_R := \lambda_{r1} \|w^* - w_{\text{avg}}\| + \lambda_{r2} \sum_i \|\delta_i\|$$

(Equation 4). Two distinct roles:

The mean-latent term keeps the optimised code from wandering into low-density regions of $\mathcal{W}$, where the generator produces artefacts. It is the same prior as the truncation trick, applied as a soft penalty during optimisation rather than as a hard projection at sampling time. w_{avg} is computed “by randomly sampling a large number of latents in $\mathcal{Z}$ space, mapping them to $\mathcal{W}$ space, and computing the average.”

The offset term implements the $\mathcal{W}+$ decomposition described in Section 3.2. The paper explains: “we decompose the w^+ latent into a single base w^* latent and n offset latents δ_i . The latent used for layer i is $w + \delta_i$. We use the L_2 norm as regularizer to ensure that the δ_i s remain small.” This is the mechanism that buys $\mathcal{W}+$ ’s expressiveness while paying only a controlled amount of its realism penalty. The authors add: “Based on our visual analysis of the results, we use larger weights for the body generator than the face generator for this regularizer” — the body is the thing that must stay realistic; the face is the thing that must change to match the user.

9.6 Loss terms 4 and 5: anchoring the two halves

Depending on which half of the composite must be preserved, one or both of two reconstruction anchors are added.

Body preservation (Equation 6). When optimising w_A we do not want the body to drift away from the body we started with, so we anchor the region *outside* the face box to a fixed reference body I_{ref} :

$$\mathcal{L}_{RB} := \lambda_5 \mathcal{L}_1(R^O(I_A), R^O(I_{\text{ref}})) + \lambda_6 \mathcal{L}_{\text{LPIPS}}(R^O(I_A), R^O(I_{\text{ref}}))$$

Face preservation (Equation 8). Symmetrically, when optimising w_B we must not lose the user’s identity, so we anchor the *interior* of the face crop to the reference face:

$$\mathcal{L}_{RF} := \lambda_7 \mathcal{L}_1(R^I(I_B), R^I(I_{\text{ref}})) + \lambda_8 \mathcal{L}_{\text{LPIPS}}(R^I(I_B), R^I(I_{\text{ref}}))$$

The paper notes: “For more precise control, face segmentation can be used instead of bounding boxes.”

Observe the complementary masks. R^O is everything outside the face box; R^I is the interior of the face box. Together with the border frame E_8 , they partition the image into three regulatory zones: *keep the body*, *keep the face*, *make the seam agree*. The optimisation is free only in the narrow band between them — which is exactly where the physical blending must happen (neck, jawline, hair fall onto shoulders).

9.7 The four application modes

The paper mixes and matches these terms depending on the task.

Mode 1 — Face refinement / face swap. Body fixed, improve its face. Keep w_A fixed, optimise only w_B (Equation 5):

$$\min_{w_B} (\mathcal{L}_A + \mathcal{L}_B).$$

The paper notes this “almost produces satisfactory results” but “boundary discontinuities show up at times.”

Mode 2 — Joint refinement with body preservation. Optimise both, but keep the body appearance unchanged (Equation 7):

$$\min_{w_A, w_B} (\mathcal{L}_A + \mathcal{L}_B + \mathcal{L}_R + \mathcal{L}_{RB}).$$

Mode 3 — Body generation for an existing face. This is *this project’s* mode. Given a real face inverted to w_B , find a body that suits it. The face identity must be preserved, so $\mathcal{L}_{RF}$ is added (Equation 9):

$$\min_{w_A, w_B} (\mathcal{L}_A + \mathcal{L}_B + \mathcal{L}_R + \mathcal{L}_{RF})$$

The authors describe the intent precisely: optimise w_A “such that $G_A(w_A)$ produces a body that looks compatible with the input face in terms of pose, skin tone, gender, hair style, etc.”, while discouraging large changes in w_B “such that the face identity is mostly preserved but the boundary and background pixels can be slightly adjusted to make the optimization of w_A easier.”

Mode 4 — Face-body montage. Combine a specific face with a specific body; both must be preserved (Equation 10):

$$\min_{w_A, w_B} (\mathcal{L}_A + \mathcal{L}_B + \mathcal{L}_R + \mathcal{L}_{RF} + \mathcal{L}_{RB}).$$

A striking observation the authors report for Mode 4: “While we do not have any explicit loss encouraging skin tone coherence, given faces with different skin tones, our joint optimization slightly adjusts the skin tone of the body’s neck and hand pixels to minimize appearance incoherence and boundary discrepancy in the final results.” Skin-tone harmonisation is an *emergent* consequence of the coarse and boundary losses acting through a disentangled latent space. There is no skin-tone term anywhere in the objective.

9.8 The alternating optimisation schedule

The moving-boundary difficulty flagged in Section 2.3 is now addressed. The authors state it plainly:

One challenge in the joint optimization of w_A and w_B is that the boundary condition Ω depends on the variable w_A . We address this by alternately optimizing for w_A and w_B , and reevaluating the boundary after each update of w_A . We stop the process when the updates converge.

Schematically:

repeat:

```

    for k steps:  fix w_A, take gradient steps on w_B  (face phase)
    for k steps:  fix w_B, take gradient steps on w_A  (body phase)
                  periodically re-run the face detector on G_A(w_A)
                  to update the crop box Omega

```

until converged

This is block coordinate descent on (w_A, w_B) with the constraint set refreshed between blocks. Freezing w_A during the face phase makes Ω constant, so the face sub-problem is a well-posed smooth optimisation on a fixed domain. Freezing w_B during the body phase means the target the body must match is constant, so the body sub-problem is likewise well-posed. Only between blocks does the domain move.

9.9 Why gradients can flow through two frozen, independently trained generators

This deserves an explicit answer, because it is the conceptual crux.

Nothing in the construction requires the two networks to share weights, share a latent space, or ever to have seen the same data. What is required is only that each is a **differentiable map** from its latent space to pixels. Given that, the composite loss $\mathcal{L}(G_A(w_A), G_B(w_B))$ is a differentiable scalar function of (w_A, w_B) , and

$$\frac{\partial \mathcal{L}}{\partial w_B} = \underbrace{\frac{\partial \mathcal{L}}{\partial I_B}}_{\text{from } L_1 + \text{LPIPS}} \cdot \underbrace{\frac{\partial G_B(w_B)}{\partial w_B}}_{\text{face generator Jacobian}}, \quad \frac{\partial \mathcal{L}}{\partial w_A} = \frac{\partial \mathcal{L}}{\partial I_A} \cdot \frac{\partial G_A(w_A)}{\partial w_A}.$$

The Jacobian $\partial G/\partial w$ is what encodes each generator’s learned prior: it says how appearance responds to latent perturbation, and it can only produce *realisable* image changes, because the generator’s range is (very nearly) the natural image manifold of its domain. Gradient descent on w therefore cannot produce an unrealistic image no matter what the loss asks for — the worst it can do is find a realistic image that satisfies the loss badly.

This is the deep reason the method works at all. In a pixel-space blending formulation, the optimiser has $3 \times 1024 \times 1024$ free variables and must be regularised heavily to stay realistic. In InsetGAN’s formulation, the optimiser has $2 \times 18 \times 512$ variables, and realism is *free* — it is a property of the parameterisation rather than a term in the loss. StyleGAN2’s path-length regularisation (Section 5.2) makes the parameterisation well-conditioned on top of that, ensuring gradient steps of similar magnitude produce image changes of similar magnitude.

The alignment between the two *disjoint* latent spaces is thus achieved entirely through the shared image-space objective. The face GAN never learns anything about bodies; it is simply steered, by gradient descent, toward the one face in its range that happens to blend with this body.

9.10 Reported cost and quality

The authors report the joint optimisation of two generator latents at 1024×1024 taking **about 75 seconds on a Titan RTX**, dropping to about 60 seconds when G_B has 256×256 resolution [15]. FID for the face-refinement application is essentially unchanged relative to unconditional generation (25.33 vs 26.67 on bodies at $\psi = 0.7$), indicating the joint optimisation “does not modify the distribution learned by the unconditional generator and therefore does not decrease the result diversity.” The perceptual gain that FID misses is captured by the user study: **98%** of participants preferred the joint-optimisation results over the unrefined body-GAN images, against **7%** for CoModGAN; and in **12.4%** of pairs, participants preferred even the *unrefined* generated bodies over real training photographs when given one second to look.

10 System Architecture and Code Implementation

This section walks the actual code in this repository and connects each engineering decision to the mathematics of Sections 6–9. All quotations are verbatim from the repository unless marked otherwise.

10.1 Repository layout

```
Whole-body-GAN-generator/
|- server/
|   |- Adebajji_User_Whole_Body_Generation.ipynb           # ReStyle + FastAPI
|   |- Copy_of_Optimised_User_Whole_Body_Generation.ipynb # full pipeline + API
|   |- User_Whole_Body_Generation.ipynb                   # original pipeline
|   |- User_Whole_Body_Generation_Using_PTI.ipynb         # PTI front-end
|   |- Deploying_Style_Human_Inference_in_Google_Colab_environment.ipynb
|- models/
|   |- style_human.ipynb                                   # StyleGAN-Human demos
|- user_interface/
|   |- UI/                                                  # early Swift client
|   |- Whole-Body GAN Demo/                                # Firebase + CocoaPods client
```

```

|- User Whole Body Generation/
|   |- StyleGAN-Human/                                # body GAN + insetgan.py
|   |- restyle/                                         # ReStyle encoder
|- data_cleaning/                                       # background blur/replace
|- requirements/

```

10.2 The generation pipeline end to end

```

iOS client (UIImagePickerController)
|
v  Firebase Storage: putData -> gs://whole-body-gan-demo.appspot.com/image.jpg
Colab GPU runtime
|
v  urllib.request.urlretrieve(firebase_url)
dlib CNN detector + 68-point landmarks -> aligned 256x256 face crop
|
v  ReStyle pSp, n_iters_per_batch = 5
face latent w_B in W+ (18 x 512)
|
v  body_z ~ N(0,I) from body_seed -> w_A, truncated by psi
InsetGAN.dual_optimizer (3 stages, alternating)
|
v  paste G_B(w_B*) into G_A(w_A*) at the detected crop
PNG (side-by-side before/after) + MP4 (optimisation trajectory)
|
v  FastAPI FileResponse over ngrok tunnel
iOS client displays result

```

10.3 Step 1: environment and model acquisition

Both the body and face generators are pulled at runtime. `models/style_human.ipynb` downloads the StyleGAN-Human StyleGAN2 1024 checkpoint from Google Drive and the FFHQ generator from NVIDIA's CDN:

```

MODEL_PATHS = {
    "stylegan2_1024": {"id": "1FlAb1rYa0r--Zj_ML8e6shmaF28hQb5",
                      "name": "stylegan2_1024.pkl"},
}

```

```

wget -q --show-progress \
    https://nvlabs-fi-cdn.nvidia.com/stylegan2-ada-pytorch/pretrained/ffhq.pkl \
    -O pretrained_models/ffhq.pkl

```

together with the two dlib models — `mmod_human_face_detector.dat` (CNN face detector) and `shape_predictor_68_face_landmarks.dat` (landmark predictor). `stylegan2_1024.pkl` is 362 MB, `ffhq.pkl` is 364 MB.

The notebooks also carry a substantial compatibility layer, because the StyleGAN2 CUDA custom operators were written against PyTorch 1.x. The patch cell rewrites `torch_utils/custom_ops.py` so that the module returned by `torch.utils.cpp_extension.load()` is used directly rather

than re-imported via `importlib` (which no longer registers in `sys.modules` under PyTorch 2.x), replaces `PIL.Image.ANTIALIAS` with `PIL.Image.LANCZOS` (removed in Pillow 10.0), rewrites `x.type().is_cuda()` to `x.is_cuda()` in the `op_edit` C++ sources, and strips the `ATen/cuda/CUDAApplyUtils.cuh` include removed in PyTorch 2.0. Ninja is installed first because those operators are JIT-compiled on first use.

10.4 Step 2: face alignment

Face alignment is not cosmetic — it is a hard requirement of the inversion step. StyleGAN2-FFHQ was trained on images normalised by the FFHQ alignment procedure (a similarity transform derived from the 68 landmarks that fixes eye positions and image scale). An unaligned face is *out of distribution* for the generator, and the encoder will invert it poorly.

```
def run_alignment(image_path):
    import dlib
    from scripts.align_faces_parallel import align_face
    predictor = dlib.shape_predictor(landmark_path)
    aligned = align_face(filepath=image_path, predictor=predictor)
    print('Aligned image size:', aligned.size)
    return aligned
```

The observed output is `Aligned image size: (256, 256)`.

10.5 Step 3: ReStyle inversion

```
encoder_type = 'psp'
restyle_experiment_args = {
    'model_path': os.path.join(pretrained_model_dir, f'restyle_{encoder_type}_ffhq_encode.pt'),
    'transform': transforms.Compose([
        transforms.Resize((256, 128)),
        transforms.ToTensor(),
        transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5]))
}]
```

```
opts.n_iters_per_batch = 5
opts.resize_outputs = False
```

```
def get_latent_code(image_file_path):
    original_image = Image.open(image_file_path).convert("RGB")
    input_image = run_alignment(image_file_path)
    img_transforms = restyle_experiment_args['transform']
    transformed_image = img_transforms(input_image)
    with torch.no_grad():
        avg_image = get_avg_image(restyle_net)
        result_batch, result_latents = run_on_batch(
            transformed_image.unsqueeze(0).cuda(), restyle_net, opts, avg_image)
    return result_latents
```

`get_avg_image` produces $\hat{y}_0 = G(\bar{w})$, the initialisation of the ReStyle recursion (Section 7.2):

```
def get_avg_image(net):
    avg_image = net(net.latent_avg.unsqueeze(0),
                    input_code=True, randomize_noise=False,
                    return_latents=False, average_code=True)[0]
    return avg_image.to('cuda').float().detach()
```

The (256, 128) resize is a portrait-aspect adaptation; the notebook records the reason explicitly: “Using (256, 256) causes a dimension mismatch when `run_on_batch` concatenates the input with the average image along the channel axis” — a direct consequence of Equation (1) of ReStyle, $x_t := x \parallel \hat{y}_t$, which requires the target and reconstruction to have identical spatial shape.

10.6 Step 4: the InsetGAN module

The reference implementation lives at `User Whole Body Generation/StyleGAN-Human/insetgan.py`. Its constructor loads both generators and freezes them:

```
config = {"latent" : 512, "n_mlp" : 8, "channel_multiplier": 2}
self.body_generator = bodyGAN(size=1024, style_dim=config["latent"],
                              n_mlp=config["n_mlp"],
                              channel_multiplier=config["channel_multiplier"])
self.body_generator.load_state_dict(torch.load(stylebody_ckpt)['g_ema'])
self.body_generator.eval().requires_grad_(False).cuda()

self.face_generator = FaceGAN(size=1024, style_dim=config["latent"], ...)
self.face_generator.load_state_dict(torch.load(styleface_ckpt)['g_ema'])
self.face_generator.eval().requires_grad_(False).cuda()

self.lpip_loss = LPIPS(net='alex').cuda().eval()
self.l1_loss = torch.nn.L1Loss(reduction='mean')
```

Note `['g_ema']` — the exponential moving average of the generator weights, which StyleGAN2 uses for inference. `requires_grad_(False)` is the code-level statement of Section 9.9: no parameter receives a gradient; only the latent tensors do.

10.6.1 The loss terms as implemented

Each paper equation from Section 9 has a direct counterpart:

Paper term	Method in <code>insetgan.py</code>	Regions compared
$\mathcal{L}_A$ (Eq. 2)	<code>loss_coarse</code>	both faces at 64×64
$\mathcal{L}_B$ (Eq. 3)	<code>loss_border</code>	8-px border frame, full res
$\mathcal{L}_{RB}$ (Eq. 6)	<code>loss_body</code>	body outside the face box
$\mathcal{L}_{RF}$ (Eq. 8)	<code>loss_face</code>	interior of the face box
$\mathcal{L}_R$ (Eq. 4)	<code>loss_reg</code>	latent-space priors

```
def loss_coarse(self, A_face, B, p1=500, p2=0.05):
    A_face = F.interpolate(A_face, size=(64, 64), mode='area')
    B = F.interpolate(B, size=(64, 64), mode='area')
```

```

loss_l1 = p1 * self.l1_loss(A_face, B)
loss_lpips = p2 * self.lpips_loss(A_face, B)
return loss_l1 + loss_lpips

```

This is $\mathcal{L}_A = \lambda_1 \mathcal{L}_1(D_{64}(\cdot), D_{64}(\cdot)) + \lambda_2 \mathcal{L}_{\text{LPIPS}}(\cdot, \cdot)$ exactly, with mode='area' performing the D_{64} downsample.

```

@staticmethod
def get_border_mask(A, x, spec):
    mask = torch.zeros_like(A)
    mask[:, :, :x, ] = 1
    mask[:, :, -x:, ] = 1
    mask[:, :, :, :x] = 1
    mask[:, :, :, -x:] = 1
    return mask

def loss_border(self, A_face, B, p1=10000, p2=2, spec=None):
    mask = self.get_border_mask(A_face, 8, spec)
    loss_l1 = p1 * self.l1_loss(A_face*mask, B*mask)
    loss_lpips = p2 * self.lpips_loss(A_face*mask, B*mask)
    return loss_l1 + loss_lpips

```

`get_border_mask(A, 8, ...)` constructs E_8 — a frame of width 8 set to one on all four edges, zero in the interior. The hard-coded 8 is the paper's E_8 notation made literal.

```

@staticmethod
def get_body_mask(A, crop, padding=4):
    mask = torch.ones_like(A)
    mask[:, :, crop[1]-padding:crop[3]+padding, crop[0]-padding:crop[2]+padding] = 0
    return mask

def loss_body(self, A, B, crop, p1=9000, p2=0.1):
    padding = int((crop[3] - crop[1]) / 20)
    mask = self.get_body_mask(A, crop, padding)
    ...

def loss_face(self, A, B, crop, p1=5000, p2=1.75):
    mask = 1 - self.get_body_mask(A, crop)
    ...

```

R^O is `get_body_mask` (ones everywhere except a padded face box) and R^I is its complement, `1 - get_body_mask`. The padding of (crop height)/20 around the box in `loss_body` deliberately excludes a margin from the body-preservation constraint, leaving the optimiser free to adjust exactly the transition zone — neck, jaw, hair-on-shoulder — where blending must physically occur.

```

def loss_reg(self, w, w_mean, p1, w_plus_delta=None, p2=None):
    return p1 * torch.mean((w - w_mean) ** 2) + p2 * torch.mean(w_plus_delta ** 2)

```

This is $\mathcal{L}_R = \lambda_{r1} \|w^* - w_{\text{avg}}\| + \lambda_{r2} \sum_i \|\delta_i\|$, as a mean of squares rather than a norm.

10.6.2 The $\mathcal{W}+$ decomposition in code

```
body_w_mean = self.body_generator.mean_latent(10000).detach()
face_w_opt  = face_w.clone().detach().requires_grad_(True)
body_w_opt  = body_w.clone().detach().requires_grad_(True)
face_w_delta = torch.zeros_like(face_w.repeat([1, 18, 1])).requires_grad_(True)
body_w_delta = torch.zeros_like(body_w.repeat([1, 18, 1])).requires_grad_(True)
```

Four optimisation variables: two base codes w^* and two offset tensors $\{\delta_i\}_{i=1}^{18}$, the offsets initialised at zero so the optimisation starts exactly on the $\mathcal{W}$ diagonal. Assembly happens in `forward`:

```
if face_w_opt.shape[1] != 18:
    face_ws = (face_w_opt).repeat([1, 18, 1])
else:
    face_ws = face_w_opt.clone()
face_ws = face_ws + face_w_delta
synth_face, _ = self.face_generator([face_ws], input_is_latent=True, randomize_noise=False)

body_ws = (body_w_opt).repeat([1, 18, 1])
body_ws = body_ws + body_w_delta
synth_body, _ = self.body_generator([body_ws], input_is_latent=True, randomize_noise=False)
```

`randomize_noise=False` is the fixed-noise requirement of Section 6.2. `input_is_latent=True` bypasses the mapping network — the optimisation lives in $\mathcal{W}/\mathcal{W}+$, never in $\mathcal{Z}$.

10.6.3 The moving crop

```
if update_crop:
    old_r = (body_crop[3]-body_crop[1]) // 2, (body_crop[2]-body_crop[0]) // 2
    _, body_crop, _ = self.detect_face_dlib(synth_body)
    center = (body_crop[1] + body_crop[3]) // 2, (body_crop[0] + body_crop[2]) // 2
    body_crop = (center[1] - old_r[1], center[0] - old_r[0],
                center[1] + old_r[1], center[0] + old_r[0])

synth_body_face = synth_body[:, :, body_crop[1]:body_crop[3], body_crop[0]:body_crop[2]]

if synth_face.shape[2] > body_crop[3]-body_crop[1]:
    synth_face_resize = F.interpolate(synth_face, size=(body_crop[3]-body_crop[1],
                                                         body_crop[2]-body_crop[0]), mode='area')
```

This is the implementation of “reevaluating the boundary after each update of w_A ” from Section 9.8. Two details are notable. The detector is re-run on the *current* body image, but only the **centre** of the new box is adopted — the half-extents `old_r` are retained. Keeping the box size fixed keeps the tensor shapes of E_8 , R^I , R^O and the interpolated face constant across steps, so the loss landscape does not develop discontinuities from resampling. The 1024^2 face image is then area-downsampled to the box size, which is typically a few hundred pixels for a full-body frame.

10.6.4 The three-stage schedule

`dual_optimizer` runs three stages, not two.

Stage 1 — face pre-conditioning (25 steps, face optimiser only). The notebook comment describes it as “remove background of face image”: the FFHQ face is generated in isolation against its own background, which must be brought into rough agreement with the body’s background before the seam constraint can be satisfied.

```
loss_face    = self.loss_face(synth_face_raw, ref_face, face_crop, 5000, 1.75)
loss_coarse  = self.loss_coarse(synth_face, synth_body_face, 50, 0.05)
loss_border  = self.loss_border(synth_face, synth_body_face, 1000, 0.1)
loss = loss_coarse + loss_border + loss_face
```

Learning rate is halved (`face_initial_learning_rate / 2`), and the coarse weight is only 50 — a deliberately gentle start.

Stage 2 — body search (150 steps, body optimiser only).

```
update_crop = True if (step % 50 == 0) else False
...
loss_coarse  = self.loss_coarse(synth_face, synth_body_face, 500, 0.05)
loss_border  = self.loss_border(synth_face, synth_body_face, 2500, 0)
loss_body    = self.loss_body(synth_body, ref_body, body_crop, 9000, 0.1)
loss_reg     = self.loss_reg(body_w_opt, body_w_mean, 15000, body_w_delta, 0)
loss = loss_coarse + loss_border + loss_body + loss_reg
```

This is Mode 3 of Section 9.7 in its body phase: find a body compatible with the fixed face. The crop is re-detected every 50 steps.

Stage 3 — joint alternating optimisation (joint_steps, default 500).

```
interval = 50
joint_face_steps = joint_steps // 2
joint_body_steps = joint_steps // 2
flag = -1
for step in pbar:
    if step % interval == 0: flag += 1
    text_flag = 'optimize_face' if flag % 2 == 0 else 'optimize_body'
```

The alternation block size is 50 steps, and the total budget is split evenly between the two phases. In the face phase:

```
loss_face    = self.loss_face(synth_face_raw, ref_face, face_crop, 5000, 1.75)
loss_coarse  = self.loss_coarse(synth_face, synth_body_face, 500, 0.05)
loss_border  = self.loss_border(synth_face, synth_body_face, 25000, 0)
loss = loss_coarse + loss_border + loss_face
```

and in the body phase:

```
loss_coarse  = self.loss_coarse(synth_face, synth_body_face, 500, 0.05)
loss_border  = self.loss_border(synth_face, synth_body_face, 2500, 0)
loss_body    = self.loss_body(synth_body, ref_body, body_crop, 9000, 0.1)
```

```
loss_reg      = self.loss_reg(body_w_opt, body_w_mean, 25000, body_w_delta, 0)
loss = loss_coarse + loss_border + loss_body + loss_reg
```

Collecting the actual weights:

Term	Stage 1	Stage 2	Stage 3 face	Stage 3 body
λ_1 (coarse L_1)	50	500	500	500
λ_2 (coarse LPIPS)	0.05	0.05	0.05	0.05
λ_3 (border L_1)	1000	2500	25000	2500
λ_4 (border LPIPS)	0.1	0	0	0
λ_5 (body L_1)	–	9000	–	9000
λ_6 (body LPIPS)	–	0.1	–	0.1
λ_7 (face L_1)	5000	–	5000	–
λ_8 (face LPIPS)	1.75	–	1.75	–
λ_{r1} (mean latent)	–	15000	–	25000
λ_{r2} (offsets)	–	0	–	0

Four observations, the last two of which are deviations from the published method.

The seam weight escalates sharply. The border L_1 weight runs $1000 \rightarrow 2500 \rightarrow 25,000$ across the stages, a factor of 25 from first to last. The schedule progressively converts seamlessness from a soft preference into a near-hard constraint, once the body has already been manoeuvred into roughly the right configuration.

The L_1 and LPIPS weights are not directly comparable. They differ by three to five orders of magnitude, but this is largely a normalisation artefact rather than a statement of relative importance. Two separate effects are at work. For the three masked terms (`loss_border`, `loss_body`, `loss_face`), the L_1 mean is taken over the *whole* tensor after multiplication by a mask that is zero over most of its area, so the reported value is diluted by the zeroed region and its raw magnitude is very small. For the unmasked `loss_coarse`, no such dilution occurs, and the 10^4 ratio between λ_1 and λ_2 does reflect a genuine decision to let the pixel term drive the optimisation with LPIPS acting only as a perceptual corrective — consistent with the authors’ stated preference for L_1 .

$\lambda_{r2} = 0$ **throughout.** The reference implementation **disables the δ regulariser**, keeping only the mean-latent prior. The $\mathcal{W}+$ offsets are therefore unconstrained in this code path, which is a deviation from the paper’s Equation (4).

$\mathcal{L}_R$ is applied only to the body latent. `loss_reg` is called exclusively with `body_w_opt` and `body_w_delta`; the face code is never regularised toward w_{avg} in any of the three stages. The paper’s remark that “we use larger weights for the body generator than the face generator for this regularizer” implies a non-zero weight on both; this implementation takes the face weight to zero. In this application that is defensible — the face code comes from an inversion of a real photograph and pulling it toward the FFHQ mean would erode exactly the identity the system is trying to preserve — but it is a departure from Equation (4) as written, and it means face realism is maintained only by $\mathcal{L}_{RF}$ and by the generator’s own prior.

10.6.5 Optimisers and learning-rate schedule

```
face_optimizer = torch.optim.Adam([face_w_opt, face_w_delta], betas=(0.9, 0.999),
                                  lr=face_initial_learning_rate) # 0.02
body_optimizer = torch.optim.Adam([body_w_opt, body_w_delta], betas=(0.9, 0.999),
                                  lr=body_initial_learning_rate) # 0.05
```

Two independent Adam instances, one per block of the coordinate descent, each owning that block’s base code and offsets. Separate optimisers matter: Adam’s first- and second-moment estimates would otherwise be corrupted by the long stretches during which a variable receives no gradient.

```
def update_lr(init_lr, step, num_steps, lr_rampdown_length, lr_rampup_length):
    t = step / num_steps
    lr_ramp = min(1.0, (1.0 - t) / lr_rampdown_length)
    lr_ramp = 0.5 - 0.5 * np.cos(lr_ramp * np.pi)
    lr_ramp = lr_ramp * min(1.0, t / lr_rampup_length)
    lr = init_lr * lr_ramp
    return lr
```

with `lr_rampup_length=0.05`, `lr_rampdown_length=0.25`. Writing $t = \text{step}/\text{num_steps}$, the schedule is

$$\eta(t) = \underbrace{\eta_0 \cdot \min\left(1, \frac{t}{0.05}\right)}_{\text{linear warm-up}} \cdot \underbrace{\left[\frac{1}{2} - \frac{1}{2} \cos\left(\pi \min\left(1, \frac{1-t}{0.25}\right)\right)\right]}_{\text{cosine ramp-down}}.$$

The warm-up over the first 5% avoids a large first step from a zero-initialised offset tensor; the cosine ramp-down over the final 25% anneals to zero for a clean stop. This is the StyleGAN2 projector schedule, reused here.

10.6.6 Composition and output

```
new_face_img, _ = insgan.face_generator([optim_face_w], input_is_latent=True)
new_shape = crop[3] - crop[1], crop[2] - crop[0]
new_face_img_crop = F.interpolate(new_face_img, size=new_shape, mode='area')
seamless_body, _ = insgan.body_generator([optim_body_w], input_is_latent=True)
seamless_body[:, :, crop[1]:crop[3], crop[0]:crop[2]] = new_face_img_crop
temp = torch.cat([cp_body, seamless_body], dim=3)
visual(temp, f"{outdir}/{face_seed:04d}_{body_seed:04d}.png")
```

The final composite is a single tensor slice assignment — no feathering, no Poisson blending. All blending work was done in latent space by $\mathcal{L}_B$. `cp_body` is the naive paste taken *before* optimisation, and the two are concatenated horizontally so the output PNG is a before/after pair. If `--video 1`, every intermediate step is dumped as a JPEG and assembled:

```
ffmpeg_cmd = (f"ffmpeg -hide_banner -loglevel error "
              f"-i {outdir}/{face_seed:04d}_{body_seed:04d}/%04d.jpg "
              f"-c:v libx264 -vf fps=30 -pix_fmt yuv420p "
              f"{outdir}/{face_seed:04d}_{body_seed:04d}.mp4")
```

10.6.7 The CLI entry point

```
python insetgan.py --face_seed=9 --body_seed=89180 \
    --joint_steps=500 --outdir outputs/insetgan --video 1
```

with defaults `--face_network=./pretrained_models/ffhq.pkl`, `--body_network=./pretrained_models/stylegan2_1024.pkl`, `--trunc=0.6`. Observed console output confirms the LPIPS backbone and the stage-1 progress bar:

```
Setting up [LPIPS] perceptual loss: trunk [alex], v[0.1], spatial [off]
Loading model from: .../lpips/weights/v0.1/alex.pth
face: 24.0000, lr: ...
```

10.7 Step 5: adapting InsetGAN to a real user photo

The stock `main()` samples *both* latents from seeds. To use a real face, the notebooks replace the face-seed branch with the encoder output. The comment in `Copy_of_Optimised_User_Whole_Body_Generation.ipynb` states the modification directly: “This is part of Main function code in Insetgan implementation. This is modified such that face seed implementation is removed. Instead `face_latent_code` is used as input.”

```
def user_joint_optimisation(face_latent_codes, image_name, body_seed, joint_steps, trunc):
    face_network = "./pretrained_models/ffhq.pkl"
    body_network = "./pretrained_models/stylegan2_1024.pkl"
    ...
    insgan = InsetGAN(body_network, face_network)
    face_mean = insgan.face_generator.mean_latent(3000)
    face_w = torch.Tensor(face_latent_codes[0][4]).cuda().unsqueeze(0)
    face_w = truncation_psi * face_w + (1-truncation_psi) * face_mean
    face_img, _ = insgan.face_generator([face_w], input_is_latent=True)

    body_z = np.random.RandomState(body_seed).randn(1, 512).astype(np.float32)
    body_mean = insgan.body_generator.mean_latent(3000)
    body_w = insgan.body_generator.get_latent(torch.from_numpy(body_z).to(device))
    body_w = truncation_psi * body_w + (1-truncation_psi) * body_mean
    body_img, _ = insgan.body_generator([body_w], input_is_latent=True)

    _, body_crop, _ = insgan.detect_face_dlib(body_img)
    face_img = F.interpolate(face_img, size=(body_crop[3]-body_crop[1],
                                             body_crop[2]-body_crop[0]), mode='area')

    cp_body = body_img.clone()
    cp_body[:, :, body_crop[1]:body_crop[3], body_crop[0]:body_crop[2]] = face_img

    optim_face_w, optim_body_w, crop = insgan.dual_optimizer(
        face_w, body_w, joint_steps=joint_steps,
        seed=f'{image_filename_short}_{body_seed:04d}',
        output_path=outdir, video=video)
```

Three things to note. `face_latent_codes[0][4]` extracts the fifth (final) ReStyle iterate for the

first image in the batch — the converged w_5 of Section 7.5. Truncation is applied to the *inverted* code, which is a meaningful trade-off: it pulls the user’s identity toward the FFHQ mean, improving realism and blendability at the cost of identity fidelity; the notebooks call it with `trunc = 0.6` in the PTI path and `trunc = 1` (no truncation) in the API defaults. And the body is still seeded randomly, which is what supplies property (P5), diversity — the same face with different `body_seed` values yields different bodies, matching the paper’s multimodal body generation.

The PTI variant is the same function with a `joint_optimization` flag threaded through, allowing the dual optimiser to be skipped entirely for speed:

```
file_paths = user_body_optimisation(
    face_latent_codes=face_latent_codes[0], image_name=image_filename,
    body_seed=221, joint_optimization=False, joint_steps=500, trunc=0.6)
```

10.8 Step 6: the FastAPI serving layer

Three notebooks expose HTTP APIs, and they differ. It is worth recording what each actually implements rather than what the README summarises.

`Deploying_Style_Human_Inference_in_Google_Colab_environment.ipynb` — the demo-oriented API over the four StyleGAN-Human capabilities. It first mechanically strips the click decorators from the CLI scripts to turn them into importable callables:

```
for src_name, func_to_expose in [('generate.py', None), ('edit.py', None),
                                ('style_mixing.py', None), ('insetgan.py', None)]:
    src = pathlib.Path(f'/content/StyleGAN-Human/{src_name}')
    dst = pathlib.Path(f'/content/StyleGAN-Human/{src_name.replace(".py", "1.py")}')
    text = re.sub(r'@click\.[^\n]+\n', '', src.read_text())
    text = text.replace('    ctx: click.Context,\n', '')
    dst.write_text(text)
```

producing `generate1.py`, `edit1.py`, `style_mixing1.py`, `insetgan1.py`. The routes are then:

Method	Path	Backing call
GET	/	health check, returns {"Hello": "World"}
GET	/generate_single_image	generate_images(...), returns one PNG
GET	/generate_image	generate_images(...), returns a ZIP of PNGs
GET	/upper_length_Edit	editmain(attr_name="upper_length", ...), ZIP of MP4s
GET	/bottom_length_Edit	editmain(attr_name="bottom_length", ...), ZIP of MP4s
GET	/Style_Mixing_EndPoint	generate_style_mix(...), ZIP containing grid.png
GET	/Joint_Optimisation_Endpoint	insmain(...), ZIP of PNG + MP4

```
@app.get("/Joint_Optimisation_Endpoint")
def joint_optimisation_endpoint(face_seed='9', body_seed='89180',
                                joint_steps='30', trunc='1'):
    insmain(face_network="pretrained_models/ffhq.pkl",
            body_network="./pretrained_models/stylegan2_1024.pkl",
            face_seed=int(face_seed), body_seed=int(body_seed),
            joint_steps=int(joint_steps), truncation_psi=float(trunc),
            outdir='outputs/insetgan', video=1)
```

Multi-file responses are streamed as an in-memory ZIP:

```
def zipfile_new(file_list):
    zip_io = BytesIO()
    with zf.ZipFile(zip_io, mode='w', compression=zf.ZIP_DEFLATED) as zip:
        for fpath in file_list:
            zip.write(fpath)
    return StreamingResponse(iter([zip_io.getvalue()]),
                               media_type="application/x-zip-compressed",
                               headers={"Content-Disposition": "attachment;filename=final_archive.zip"})
```

Copy_of_Optimised_User_Whole_Body_Generation.ipynb — the production API for the user-photo path, with CORS enabled for browser clients:

Method	Path	Behaviour
GET	/join-optimisation-url-endpoint	download face from <code>image_url</code> , invert, run InsetGAN, return MP4
POST	/join-optimisation-upload-endpoint	accept multipart/form-data upload, same pipeline
GET	/generate_single_image	unconditional body from a seed
GET	/video	stream the most recent MP4 (HTTP 206 with Content-Range)
GET	/html	Jinja2 preview page
GET	/docs	FastAPI Swagger UI

```
@app.get('/join-optimisation-url-endpoint')
async def joint_optimisation_endpoint(image_url: str, body_seed: str = '20345',
                                       joint_steps: str = '5', trunc: str = '1'):
    global video_path
    local_img = os.path.join('/content/User-Whole-Body-Generation', 'up_image.jpg')
    urllib.request.urlretrieve(image_url, 'test.jpg')
    Image.open('test.jpg').save(local_img)
    face_latent_codes = get_latent_code(local_img)
    filepaths = user_joint_optimisation(
        face_latent_codes, 'up_image.jpg', body_seed, joint_steps, trunc)
```

```
video_path = filepaths[1]
return FileResponse(path=filepaths[1], media_type='video/mp4', filename='result.mp4')
```

The `image_url` parameter is the hinge of the whole client-server design: it is the Firebase Storage download URL that the iOS app has just written to.

Adebanji_User_Whole_Body_Generation.ipynb — the minimal variant, exposing only GET / and GET /image/{filename}.

All three are hosted identically: uvicorn in a daemon thread so the notebook cell returns, with pyngrok providing the public URL.

```
ngrok.set_auth_token('...')
public_url = ngrok.connect(8000).public_url
nest_asyncio.apply()
_server = threading.Thread(target=uvicorn.run,
    kwargs={'app': app, 'host': '0.0.0.0', 'port': 8000}, daemon=True)
_server.start()
```

`nest_asyncio.apply()` is required because uvicorn wants to own an event loop and Jupyter already has one running.

10.8.1 Security finding: hardcoded credentials committed to the repository

This is an active security defect requiring remediation, not a stylistic observation. Live ngrok authentication tokens are committed in plaintext to version control. An audit of the repository finds **two distinct tokens across five notebook files**:

Token	File
Token A (29LR1RD7...)	server/Adebanji_User_Whole_Body_Generation.ipynb
Token A	server/Copy_of_Optimised_User_Whole_Body_Generation...
Token A	server/User_Whole_Body_Generation.ipynb
Token A	server/Deploying_Style_Human_Inference_in_Google_Cola
Token B (29IkRnPz...)	User Whole Body Generation/StyleGAN-Human/Demo_1_Generating_Images_fr

They appear both as `ngrok.set_auth_token('...')` calls and as `!ngrok authtoken ...` shell invocations. Notably, one notebook cell already carries a remediation comment — “The ngrok token previously hardcoded here has been removed. It was committed to git history and must be REVOKED” — which means the exposure was recognised but the cleanup was only partial: the token was removed from that one cell while remaining live in four other files.

Impact. An ngrok authtoken permits an attacker to open tunnels under the victim’s ngrok account, consuming quota, impersonating the account’s endpoints, and — on paid tiers — claiming reserved domains. Because the token is in git *history*, deleting it from the working tree is insufficient.

Required remediation.

1. **Revoke both tokens immediately** at <https://dashboard.ngrok.com/tunnels/auth/tokens>. Revocation, not deletion, is the operative step: any credential pushed to a repository must

- be treated as compromised regardless of whether the repository is public.
- 2. Replace all hardcoded occurrences with an environment-variable or secrets lookup — for Colab, `google.colab.userdata.get('NGROK_TOKEN')`; otherwise `os.environ['NGROK_TOKEN']`.
- 3. Purge the values from git history (`git filter-repo`, or the GitHub support path for cached views) if the repository is or ever was public.
- 4. Add a pre-commit secret scanner (`gitleaks`, `detect-secrets`) so that notebook outputs and cells are checked before commit — notebooks are a common blind spot because credentials hide in JSON `source` arrays.

The same discipline applies to `requirements/GoogleService-Info.plist`, which carries Firebase project identifiers and API keys. These are less sensitive by design (Firebase client keys are intended to be shipped in apps and are not secrets on their own), but they are only safe if Firebase Storage security rules are actually restrictive — and as Section 12.7 notes, this deployment writes every user’s photo to a single world-readable path.

10.9 Step 7: the iOS client and the Firebase relay

The client is a UIKit application (`user_interface/Whole-Body GAN Demo/`) using CocoaPods with `Firebase/Core` and `Firebase/Storage`, targeting iOS 15.4.

The capture-and-upload flow lives in `GeneratorBaseOnPicViewController.swift`. The user chooses camera or photo library:

```
@IBAction func takePhoto(_ sender: Any) {
    let vc = UIImagePickerController()
    vc.sourceType = .camera
    vc.allowsEditing = true
    vc.delegate = self
    present(vc, animated: true)
}
```

and on selection the image is written to a fixed Firebase Storage path with a live progress observer:

```
let uploadRef = Storage.storage().reference(withPath: "image.jpg")
guard let imageData = image.jpegData(compressionQuality: 1) else { return }
let uploadMeta = StorageMetadata.init()
uploadMeta.contentType = "image/jpeg"

let taskRef = uploadRef.putData(imageData, metadata: uploadMeta) { (downloadMeta, error) in
    if let error = error { print("Cannot upload!"); return }
    print("Download Complete!")
}

taskRef.observe(.progress, handler: { [weak self] (snapshot) in
    guard let pct = snapshot.progress?.fractionCompleted else { return }
    self!.progressBar.progress = Float(pct)
    self!.progLabel.text = String(Int(Float(pct) * 100)) + "%"
})

taskRef.observe(.success, handler: { [weak self] (snapshot) in
    self!.imageFrame.image = image
    self!.b1.isEnabled = true; self!.b2.isEnabled = true; self!.b3.isEnabled = true
})
```

```
})
```

The view controller also exposes a `body_seed` text field and a `joint_steps` slider capped at `maxSteps = 1000`, which map directly onto the two most consequential InsetGAN parameters.

The Colab side then reads the uploaded file through the public Firebase download URL; the test harness in the notebook shows the exact endpoint:

```
urllib.request.urlretrieve(
    'https://firebasestorage.googleapis.com/v0/b/whole-body-gan-demo.appspot.com/o/image.jpg?alt=media&token=...',
    'test.jpg')
```

The simpler client in `user_interface/UI/` talks to the seed-based endpoint directly over the ngrok tunnel:

```
let baseURL = String("https://b047-34-133-189-72.ngrok.io/generate_single_image?seed=")
...
let url = baseURL + String(seed.text!) + String("&trunc=") + String(trunc.value)
image.downloaded(from: url)
```

with an asynchronous UIImageView extension performing the fetch (`imageView.swift`).

Why Firebase at all? Because the compute lives in a Colab notebook behind a transient ngrok URL that changes on every runtime restart, the mobile client cannot rely on a stable ingress for uploads. Firebase Storage provides a stable, authenticated, resumable-upload endpoint with a fixed public download URL, so the only volatile piece of configuration is the ngrok host used to *trigger* generation. Image bytes never traverse the tunnel on the way in — only the result comes back through it. This is a sensible workaround for a demo-grade deployment, and Section 13 discusses how the 2026 successor removes the need for it entirely.

11 Experiments and Demonstrated Capabilities

11.1 Unconditional full-body generation

The baseline capability, run in `models/style_human.ipynb`:

```
python generate.py --outdir=outputs/stylegan2_1024/ --seeds=4-10 --trunc=0.7 \
    --network=pretrained_models/stylegan2_1024.pkl --version 2
```

`--version` selects the StyleGAN generation (1, 2 or 3), since the StyleGAN-Human release ships checkpoints for all three; the notebook derives it from the experiment name (`version = experiment_type.split("_")[0][-1]`). The `--trunc` flag is the ψ of Section 3.4. The `generate.py` docstring records the intended usage of both extremes — `--trunc=1` for untruncated diversity and `--trunc=0.8` for quality — and the served API defaults to $\psi = 0.5$.

11.2 Attribute editing via latent directions

`edit.py` implements three distinct editing families over the same two attributes. From `edit/edit_config.py`:

```
attr_dict = dict(
    interface_gan={ # strength
```

```

        'upper_length': [1],
        'bottom_length': [1]
    },
    stylespace={ # layer, strength, threshold
        'upper_length': [5, 5, 0.0028],
        'bottom_length': [3, 5, 0.003]
    },
    sefa={ # layer, strength
        'upper_length': [[4, 5, 6, 7], -5],
        'bottom_length': [[4, 5, 6, 7], 5]
    }
)

```

InterFaceGAN [29] fits a linear SVM in latent space separating labelled positives from negatives for an attribute, and takes the unit normal n of the separating hyperplane as the edit direction:

$$w' = w + \alpha n.$$

Conditional manipulation of a target attribute while preserving a correlated one is achieved by projecting out the interfering normal, $n_1 - (n_1^\top n_2)n_2$.

StyleSpace [31] operates not in $\mathcal{W}$ but in $\mathcal{S}$, the space of per-layer *channel-wise style parameters* $s_\ell = A_\ell(w)$. Wu, Lischinski and Shechtman showed $\mathcal{S}$ is substantially more disentangled than $\mathcal{W}$ or $\mathcal{W}+$, with individual channels controlling remarkably localised attributes. The config’s [layer, strength, threshold] triple selects a layer, a perturbation magnitude, and a gradient-magnitude threshold for choosing which channels in that layer to touch. This is why `edit_helper.py` reimplements the modulated convolution by hand (`conv_warper`, Section 6.3) — it needs to intercept s_ℓ between the affine transform and the convolution, which the stock generator does not expose.

SeFa [30] requires no labels at all. It factorises the first affine layer’s weight matrix A directly: the directions that produce the largest change in the modulated output are the top eigenvectors of $A^\top A$, obtained in closed form. The config’s [[4, 5, 6, 7], -5] applies the discovered direction to layers 4–7 with a signed strength; the opposite signs for `upper_length` and `bottom_length` indicate they lie along the same discovered axis in opposite directions.

The invocations run in the notebooks:

```

python edit.py --outdir outputs/editing --network pretrained_models/stylegan2_1024.pkl \
    --attr_name upper_length --seeds 61531,61570,61571,61610

python edit.py --outdir outputs/editing --network pretrained_models/stylegan2_1024.pkl \
    --attr_name bottom_length --seeds 61531,61570,61571,61610

```

with `--gen_video True` and `--combine True` by default, so the output is an MP4 sweeping the edit strength with all three methods side by side — which is what the `/upper_length_Edit` and `/bottom_length_Edit` endpoints return. The selected seeds are not arbitrary; `edit.py` carries a commented list of curated seeds grouped by their starting attribute value.

11.3 Style mixing

```
python style_mixing.py --outdir=outputs/stylemixing \
    --rows=85,100,75,458,1500,86 --cols=55,821,1789,293,75 \
    --network=pretrained_models/stylegan2_1024.pkl --styles=0-3
```

Six row seeds and five column seeds produce a 7×6 grid (including headers) written as `grid.png`. `--styles=0-3` transfers the four coarsest style layers. Observed console output confirms the pipeline stages:

```
Loading networks from "pretrained_models/stylegan2_1024.pkl"...
Generating W vectors...
Generating images...
Generating style-mixed images...
Saving image grid...
```

11.4 InsetGAN joint optimisation

The headline experiment:

```
python insetgan.py --face_seed=9 --body_seed=89180 \
    --joint_steps=500 --outdir outputs/insetgan --video 1
```

produces `0009_89180.png` — the naive paste and the optimised composite side by side — and `0009_89180.mp4`, a 30 fps rendering of all 675 intermediate frames ($25 + 150 + 500$). The video is the most informative artefact the system produces: it shows the face’s background dissolving into the body’s during stage 1, the body shifting pose and skin tone during stage 2, and the seam tightening during stage 3.

11.5 The full personalised pipeline

Driven by API call or notebook:

```
face_latent_codes = get_latent_code(local_img)          # ReStyle, 5 iterations
filepaths = user_joint_optimisation(face_latent_codes, 'up_image.jpg',
                                     body_seed, joint_steps, trunc)
```

The API defaults `joint_steps='5'` — a very short optimisation chosen for demo latency, giving stages of $25 + 150 + 5 = 180$ steps. The full-quality setting used in the notebooks is `joint_steps=500`. The PTI path is timed explicitly:

```
stime = time.time()
face_latent_codes = get_latent_code_using_PTI(path_to_original_image_folder)
...
endtime = time.time()
print(f"total time taken is {endtime-stime}")
```

11.6 Auxiliary data-cleaning experiments

`data_cleaning/blur_background.ipynb` and `change_background.ipynb` explore background manipulation of training and output images. This mirrors a design choice made by the InsetGAN au-

thors, who enlarged backgrounds by reflection padding and blurred them with a 27-pixel Gaussian kernel specifically “to focus the generator capacity on modeling only the foreground humans” [15]. They also record why they did *not* remove backgrounds entirely: segmentation masks are imperfect at boundaries, and “current GAN architectures do not handle large areas of uniform color well.”

12 Limitations and Failure Cases

12.1 Identity fidelity is bounded by the latent space

The most fundamental limitation is structural. The user’s face can only ever be represented as a point in StyleGAN2-FFHQ’s $\mathcal{W}+$ space, so the best achievable identity match is the closest point on the generator’s image manifold. For faces well covered by FFHQ, that is close. For faces that are not — unusual lighting, occlusions, headwear, demographics under-represented in FFHQ — visible distortion remains regardless of how many optimisation steps are run. The repository’s own successor project states the comparison plainly in its README:

	GAN system (2022)	Diffusion system (2026)
Identity fidelity	Good (limited by GAN latent space)	Better (ArcFace embedding conditions the UNet directly)

Three specific mechanisms compound this in the present implementation:

Truncation applied to the inverted code. In `user_joint_optimisation`, the user’s code is truncated toward the FFHQ mean: `face_w = truncation_psi * face_w + (1-truncation_psi) * face_mean`. At the notebooks’ $\psi = 0.6$, this is a 40% interpolation toward the average face — a substantial, deliberate sacrifice of identity for realism.

No identity loss during joint optimisation. As established in Section 3.6, `insetgan.py` uses only L_1 and LPIPS. Identity is anchored only by $\mathcal{L}_{RF}$, which compares against the *reference face generated from the inverted code*, not against the user’s original photograph. Any identity lost during inversion is therefore locked in; the joint optimisation can only preserve it, never recover it. Adding an ArcFace term against the original photo would be a direct, low-cost improvement.

PTI’s tuned weights are discarded. As noted in Section 8.5, the PTI path extracts the pivot code w_p but runs it through the *original* FFHQ generator, discarding θ^* . Since the whole point of PTI is that the fidelity gain lives in the weights, this forfeits most of the benefit while paying the full two-minute cost.

12.2 Seam and blending artefacts

The InsetGAN authors document these directly [15]: “the joint optimization approach may change details such as hair style, neckline or clothing details. In many cases the changes are minor, but in some cases changes can be larger.” The failure mode follows from the objective — the optimiser is free to move w_A within the constraints of $\mathcal{L}_{RB}$, and hair and collars sit precisely in the unregularised transition zone created by the $(\text{crop height})/20$ padding in `get_body_mask`.

Complex hairstyles are the hardest case, and the paper is explicit that in such cases it is preferable “to discourage large changes in w_B .” Long hair crosses the face-box boundary, so $\mathcal{L}_B$ must reconcile

two generators’ independent notions of where a hair strand goes — a constraint that cannot always be satisfied by any (w_A, w_B) .

The rectangular crop is itself a limitation. The paper notes that “for more precise control, face segmentation can be used instead of bounding boxes.” A box forces the seam through the ears, jawline and forehead-hairline regardless of the actual head silhouette.

12.3 Artefacts inherited from the body generator

InsetGAN improves the face and nothing else. The authors list what remains: “symmetry, e.g. noticeable in the hands and feet and on outfits ... and the consistency of the fabric used for the clothing.” Hands and feet in particular suffer from the same capacity-dilution argument that motivated the method for faces, and the framework’s own answer — add a specialist hands generator as a second inset — was demonstrated but not adopted here.

12.4 Dataset bias

The InsetGAN paper devotes a full subsection to this, and it applies with equal force to StyleGAN-Human. DeepFashion is roughly 9:1 female and dominated by slim fashion models; the authors’ purchased dataset over-represents young Asian women. The direct consequence, stated in the paper: “our results on other ethnicities contain more artifacts in the face region,” results have “limited variations in body type and pose,” and “since the age distribution in our data is almost exclusively adult humans, we are not able to faithfully produce bodies for faces of children.” A user-facing personalisation product built on these checkpoints will produce systematically worse results for users outside the training distribution. This is an ethical and product limitation, not only a technical one.

12.5 Inference latency

The reference implementation runs the joint optimisation in about 75 seconds on a Titan RTX [15]. This system’s end-to-end latency is considerably worse — the successor repository’s comparison table records **3–5 minutes** per generation — because it adds:

- ReStyle inversion (~10 s) or PTI inversion (~2 min);
- generator `.pkl` to `.pth` conversion on first use (`legacy.convert`);
- dlib CNN detection re-run periodically inside the optimisation loop;
- per-step JPEG dumping and a final ffmpeg encode when `--video 1`;
- a Colab T4, which is roughly a third the throughput of a Titan RTX;
- model download and CUDA JIT compilation on cold start.

The API’s `joint_steps='5'` default is a direct concession to this: it produces a visibly worse composite in exchange for tolerable latency. There is no architecture change that fixes this within the GAN-inversion paradigm — the optimisation loop is intrinsic to the method.

12.6 Serving fragility

The deployment is demo-grade and honestly so. Colab runtimes are preemptible and time-limited; every restart re-downloads roughly 750 MB of checkpoints and recompiles the CUDA extensions. The ngrok URL changes on every restart, so the iOS client’s hard-coded `baseURL` must be edited and the app rebuilt — visible in the committed literal

"https://b047-34-133-189-72.ngrok.io/generate_single_image?seed=". There is no authentication on any endpoint, no rate limiting, no request queue (a second request during an optimisation contends for the same GPU), and global mutable state (`video_path`) makes concurrent requests unsafe. The single Firebase path `image.jpg` is shared across all users, so two simultaneous users overwrite each other.

Additionally, the FastAPI routes for generation are `GET` requests that trigger minutes of GPU computation and mutate server state — a semantic misuse of the verb that also makes them vulnerable to naive caching or prefetching.

12.7 Security and privacy defects

Distinct from fragility, and more serious, the deployment has concrete security problems that would block any production use. They are collected here because they are the most actionable findings in this document.

Committed credentials (open, requires action). Two live ngrok authtokens are committed across five notebooks, as detailed in Section 10.8. Both must be revoked and replaced with a secrets lookup; removal from the working tree alone is insufficient because the values persist in git history.

No authentication or authorisation. Every FastAPI route is open to anyone who learns the ngrok URL. The generation endpoints are unauthenticated, so an arbitrary caller can occupy the single GPU indefinitely — a trivial denial-of-service — and, via `/join-optimisation-url-endpoint`, cause the server to fetch an arbitrary attacker-supplied URL. That last route is a server-side request forgery (SSRF) primitive: `urllib.request.urlretrieve` is called directly on a user-controlled `image_url` with no scheme allowlist, host validation, size cap, or timeout, so it can be pointed at internal addresses or at endpoints chosen to exhaust disk.

A shared, world-readable image path. The iOS client writes every capture to the single fixed Firebase Storage object `image.jpg`, and the Colab side reads it back through an unauthenticated public download URL. Two consequences follow: concurrent users overwrite one another's uploads, and — because the path is fixed and public — anyone who knows the bucket can read the most recent user's face photograph. For a system whose entire input is biometric data, this is the most serious defect in the deployment. Correct handling would use per-session randomised object paths, authenticated reads via Firebase security rules tied to an authenticated user, and a retention policy that deletes uploads after processing.

Unbounded uploads. `/join-optimisation-upload-endpoint` streams `file.file` to disk via `shutil.copyfileobj` with no size limit and no content type validation.

None of these are difficult to fix, and none of them were in scope for a summer-internship demonstrator. They are recorded because the document is intended as a long-term reference, and anyone tempted to reuse this serving layer needs to know that it is a demonstration harness and not a deployable service.

12.8 Reproducibility caveats

Both generator checkpoints are fetched from Google Drive file IDs and NVIDIA's CDN at runtime. Google Drive links expire and rate-limit; the notebooks contain several layers of fallback logic and comments about the virus-scan interstitial (`&confirm=t`). The Drive-hosted `ffhq.pkl` link

had already broken by the time the notebooks were re-run, requiring the NVIDIA CDN fallback. Long-term reproducibility would require vendoring checksums and a durable artefact store.

13 Future Research Directions

13.1 The GAN-to-diffusion transition, as a case study

This repository has an unusually concrete epilogue: it was superseded, in 2026, by `whole-body-diffusion-generator` which solves the same product problem with IP-Adapter FaceID [35] conditioning an SDXL [34] backbone. The comparison recorded in that repository is a compact summary of what changed in generative modelling between 2022 and 2026:

	Old (2022)	New (2026)
Model	StyleGAN-Human + ReStyle + InsetGAN joint optimisation	IP-Adapter FaceID + SDXL (single forward pass)
Face encoding	dlib alignment then ReStyle pSp (5 iterative passes)	insightface ArcFace embedding (one call)
Inference time	~3–5 min (InsetGAN optimisation loop)	~15–25 s (30 DDIM steps)
Serving	Colab + ngrok + Firebase relay	Modal serverless GPU, stable HTTPS URL
iOS integration	Upload to Firebase, then poll	Direct <code>multipart/form-data</code> POST, PNG response
Identity fidelity	Good (limited by GAN latent space)	Better (ArcFace conditions the UNet directly)

Three structural shifts are visible.

Conditioning replaced inversion. The 2022 system’s central difficulty is that a frozen unconditional generator has no input for “this person,” so identity must be laundered through a latent-space inversion. Diffusion models with cross-attention [33] accept conditioning natively; IP-Adapter adds a *decoupled* cross-attention pathway for image features alongside the text pathway, so an ArcFace embedding [27] can be injected into every UNet block. There is nothing to invert. The entire ReStyle/PTI apparatus — two research papers and roughly half of this system’s engineering — becomes unnecessary.

Optimisation replaced by a forward pass. InsetGAN’s cost is intrinsically iterative: 675 generator evaluations with backpropagation. Diffusion inference is 30 forward passes with no gradients. The order-of-magnitude latency improvement is architectural, not an implementation detail.

Compositing became unnecessary. InsetGAN exists because no single 2022 generator could do faces and bodies simultaneously at 1024^2 . SDXL, trained on billions of images with far greater capacity, can. The multi-specialist compositing framework was a brilliant answer to a capacity constraint that subsequently dissolved.

It would be wrong to read this as InsetGAN having been refuted. Its *ideas* transferred rather than expired. The general principle — coordinate several frozen pretrained models through a shared objective rather than retraining one large model — reappears throughout modern diffusion tooling (ControlNet-style adapters, LoRA composition, multi-adapter pipelines). And GANs retain

a decisive advantage in single-step sampling latency, which is why adversarial objectives returned as *distillation* targets for few-step diffusion samplers. The honest summary is that Dhariwal and Nichol’s 2021 result [38] marked the point where scaling favoured diffusion for quality, while the compositional insight of InsetGAN outlived the architecture that motivated it.

13.2 Extensions within the GAN framework

Several concrete improvements to *this* system remain unexplored and are cheap:

- **Add an identity loss to the joint optimisation.** An ArcFace cosine term between $G_B(w_B)$ and the user’s original photograph, rather than only against the inverted reference, would directly attack the primary limitation of Section 12.1.
- **Use segmentation instead of bounding boxes.** The paper suggests this explicitly; a face-parsing network would let $\mathcal{L}_B$ operate on the true head silhouette and would largely eliminate the hair-crossing failure mode.
- **Load PTI’s tuned generator.** Fixing the discarded- θ^* issue is a handful of lines and recovers most of PTI’s advantage.
- **Re-enable the δ regulariser.** $\lambda_{r2} = 0$ throughout the reference implementation; restoring it should improve realism of the optimised codes.
- **Add specialist insets for hands and feet.** The paper demonstrates a three-generator configuration (body, face, shoes); extremities are the strongest remaining artefact source.
- **Amortise the optimisation.** Train a network to predict (w_A, w_B) directly from $(w_B^{(0)}, \text{seed})$, using the InsetGAN optimisation as the supervision target — the same encoder-distillation move that turned optimisation-based inversion into pSp.

13.3 3D-aware generation

The InsetGAN authors name this as their own next step: “In future work, we propose to extend the multi-generator idea to 3D shape representations, such as 3D GANs or auto-regressive models based on transformers.” EG3D [36] demonstrated that a StyleGAN2 backbone can generate a tri-plane 3D representation rendered by volume rendering, giving multi-view-consistent synthesis with a genuine 3D prior. Extending InsetGAN’s compositing to that setting is conceptually attractive and technically hard: the seam constraint would have to hold across all viewpoints simultaneously, which turns $\mathcal{L}_B$ into an expectation over camera poses. The payoff for the present application would be substantial — a personalised avatar that can be posed and viewed from any angle rather than a single frontal image.

13.4 Video and temporal extension

Temporal consistency is the natural next axis. A naive per-frame application of InsetGAN would flicker badly: the optimisation is non-convex, the crop detector is noisy, and nothing ties consecutive solutions together. A principled extension would optimise a *trajectory* $\{(w_A^t, w_B^t)\}$ under a smoothness penalty $\sum_t \|w^{t+1} - w^t\|^2$. The style-mixing video utilities already present in the repository (`stylemixing_video.py`, `interpolation.py`) are the seed of such a capability, and the MP4 optimisation trajectories the system already produces are a degenerate instance of it.

13.5 On-device inference

Both generators are roughly 360 MB in float32 and require a CUDA GPU, so on-device inference was not remotely feasible in 2022. The situation has changed: Apple’s Neural Engine, Core ML quantisation, and distilled few-step diffusion samplers have made on-device generation practical. Removing the server removes, in one stroke, the ngrok fragility, the Firebase relay, the concurrency hazards and the privacy exposure of uploading a user’s face to third-party infrastructure. For a face-personalisation product, that last point is the strongest argument of all: the most defensible place to process a photograph of someone’s face is the device it was taken on.

13.6 Evaluation methodology

Finally, a methodological note. The InsetGAN authors are candid that FID is a poor instrument here — it “is more sensitive to result diversity than quality,” rising from 13.96 to 71.90 under truncation that visibly *improves* individual images, and their face-refinement FID differences are within noise (25.33 vs 26.67) despite a 98% user-study preference. Any future work in this area needs identity-preservation metrics (ArcFace cosine similarity to the input photograph), explicit seam metrics (gradient discontinuity across the paste boundary), and human preference studies. This repository performs none of these quantitatively, which is its clearest methodological gap.

14 References

1. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). *Generative Adversarial Nets*. Advances in Neural Information Processing Systems (NeurIPS) 27. arXiv:1406.2661.
2. Kingma, D. P., and Welling, M. (2014). *Auto-Encoding Variational Bayes*. International Conference on Learning Representations (ICLR). arXiv:1312.6114.
3. Radford, A., Metz, L., and Chintala, S. (2016). *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*. International Conference on Learning Representations (ICLR). arXiv:1511.06434.
4. Arjovsky, M., Chintala, S., and Bottou, L. (2017). *Wasserstein Generative Adversarial Networks*. International Conference on Machine Learning (ICML), PMLR 70:214–223. arXiv:1701.07875.
5. Karras, T., Aila, T., Laine, S., and Lehtinen, J. (2018). *Progressive Growing of GANs for Improved Quality, Stability, and Variation*. International Conference on Learning Representations (ICLR). arXiv:1710.10196.
6. Karras, T., Laine, S., and Aila, T. (2019). *A Style-Based Generator Architecture for Generative Adversarial Networks*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:1812.04948.
7. Karras, T., Laine, S., Aittala, M., Hellsten, J., Lehtinen, J., and Aila, T. (2020). *Analyzing and Improving the Image Quality of StyleGAN*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:1912.04958.
8. Karras, T., Aittala, M., Hellsten, J., Laine, S., Lehtinen, J., and Aila, T. (2020). *Training Generative Adversarial Networks with Limited Data*. Advances in Neural Information Processing Systems (NeurIPS) 33. arXiv:2006.06676.

9. Karras, T., Aittala, M., Laine, S., Härkönen, E., Hellsten, J., Lehtinen, J., and Aila, T. (2021). *Alias-Free Generative Adversarial Networks*. Advances in Neural Information Processing Systems (NeurIPS) 34. arXiv:2106.12423.
10. Huang, X., and Belongie, S. (2017). *Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization*. IEEE International Conference on Computer Vision (ICCV). arXiv:1703.06868.
11. Mescheder, L., Geiger, A., and Nowozin, S. (2018). *Which Training Methods for GANs do actually Converge?* International Conference on Machine Learning (ICML), PMLR 80:3481–3490. arXiv:1801.04406.
12. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., and Hochreiter, S. (2017). *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*. Advances in Neural Information Processing Systems (NeurIPS) 30. arXiv:1706.08500.
13. Fu, J., Li, S., Jiang, Y., Lin, K.-Y., Qian, C., Loy, C. C., Wu, W., and Liu, Z. (2022). *StyleGAN-Human: A Data-Centric Odyssey of Human Generation*. European Conference on Computer Vision (ECCV), Lecture Notes in Computer Science, vol. 13676, Springer. DOI: 10.1007/978-3-031-19787-1_1. arXiv:2204.11823.
14. Liu, Z., Luo, P., Qiu, S., Wang, X., and Tang, X. (2016). *DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR),
15. Frühstück, A., Singh, K. K., Shechtman, E., Mitra, N. J., Wonka, P., and Lu, J. (2022). *InsetGAN for Full-Body Image Generation*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7723–7732. arXiv:2203.07293.
16. Frühstück, A., Alhashim, I., and Wonka, P. (2019). *TileGAN: Synthesis of Large-Scale Non-Homogeneous Textures*. ACM Transactions on Graphics (SIGGRAPH), 38(4). arXiv:1904.12795.
17. Zhao, S., Cui, J., Sheng, Y., Dong, Y., Liang, X., Chang, E. I., and Xu, Y. (2021). *Large Scale Image Completion via Co-Modulated Generative Adversarial Networks*. International Conference on Learning Representations (ICLR). arXiv:2103.10428.
18. Xia, W., Zhang, Y., Yang, Y., Xue, J.-H., Zhou, B., and Yang, M.-H. (2022). *GAN Inversion: A Survey*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 45(3):3121–3138. DOI: 10.1109/TPAMI.2022.3181070. arXiv:2101.05278.
19. Abdal, R., Qin, Y., and Wonka, P. (2019). *Image2StyleGAN: How to Embed Images Into the StyleGAN Latent Space?* IEEE/CVF International Conference on Computer Vision (ICCV). arXiv:1904.03189.
20. Richardson, E., Alaluf, Y., Patashnik, O., Nitzan, Y., Azar, Y., Shapiro, S., and Cohen-Or, D. (2021). *Encoding in Style: a StyleGAN Encoder for Image-to-Image Translation*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2287–2296. arXiv:2008.00951.
21. Tov, O., Alaluf, Y., Nitzan, Y., Patashnik, O., and Cohen-Or, D. (2021). *Designing an Encoder for StyleGAN Image Manipulation*. ACM Transactions on Graphics (SIGGRAPH), 40(4). DOI: 10.1145/3450626.3459838. arXiv:2102.02766.

22. Alaluf, Y., Patashnik, O., and Cohen-Or, D. (2021). *ReStyle: A Residual-Based StyleGAN Encoder via Iterative Refinement*. IEEE/CVF International Conference on Computer Vision (ICCV), pp. 6711–6720. arXiv:2104.02699.
23. Roich, D., Mokady, R., Bermano, A. H., and Cohen-Or, D. (2022). *Pivotal Tuning for Latent-based Editing of Real Images*. ACM Transactions on Graphics, 42(1), Article 6. DOI: 10.1145/3544777. arXiv:2106.05744.
24. Menon, S., Damian, A., Hu, S., Ravi, N., and Rudin, C. (2020). *PULSE: Self-Supervised Photo Upsampling via Latent Space Exploration of Generative Models*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:2003.03808.
25. Johnson, J., Alahi, A., and Fei-Fei, L. (2016). *Perceptual Losses for Real-Time Style Transfer and Super-Resolution*. European Conference on Computer Vision (ECCV). arXiv:1603.08155.
26. Zhang, R., Isola, P., Efros, A. A., Shechtman, E., and Wang, O. (2018). *The Unreasonable Effectiveness of Deep Features as a Perceptual Metric*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:1801.03924.
27. Deng, J., Guo, J., Xue, N., and Zafeiriou, S. (2019). *ArcFace: Additive Angular Margin Loss for Deep Face Recognition*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:1801.07698.
28. Schroff, F., Kalenichenko, D., and Philbin, J. (2015). *FaceNet: A Unified Embedding for Face Recognition and Clustering*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:1503.03832.
29. Shen, Y., Gu, J., Tang, X., and Zhou, B. (2020). *Interpreting the Latent Space of GANs for Semantic Face Editing*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:1907.10786.
30. Shen, Y., and Zhou, B. (2021). *Closed-Form Factorization of Latent Semantics in GANs*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:2007.06600.
31. Wu, Z., Lischinski, D., and Shechtman, E. (2021). *StyleSpace Analysis: Disentangled Controls for StyleGAN Image Generation*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:2011.12799.
32. Ho, J., Jain, A., and Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. Advances in Neural Information Processing Systems (NeurIPS) 33. arXiv:2006.11239.
33. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:2112.10752.
34. Podell, D., English, Z., Lacey, K., Blattmann, A., Dockhorn, T., Müller, J., Penna, J., and Rombach, R. (2024). *SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis*. International Conference on Learning Representations (ICLR). arXiv:2307.01952.
35. Ye, H., Zhang, J., Liu, S., Han, X., and Yang, W. (2023). *IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models*. arXiv:2308.06721.
36. Chan, E. R., Lin, C. Z., Chan, M. A., Nagano, K., Pan, B., De Mello, S., Gallo, O., Guibas, L., Tremblay, J., Khamis, S., Karras, T., and Wetzstein, G. (2022). *Efficient Geometry-aware 3D*

- Generative Adversarial Networks*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:2112.07945.
37. Ruiz, N., Li, Y., Jampani, V., Pritch, Y., Rubinstein, M., and Aberman, K. (2023). *DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:2208.12242.
 38. Dhariwal, P., and Nichol, A. (2021). *Diffusion Models Beat GANs on Image Synthesis*. Advances in Neural Information Processing Systems (NeurIPS) 34. arXiv:2105.05233.
 39. King, D. E. (2009). *Dlib-ml: A Machine Learning Toolkit*. Journal of Machine Learning Research, 10:1755–1758.
 40. Cao, Z., Hidalgo, G., Simon, T., Wei, S.-E., and Sheikh, Y. (2021). *OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 43(1):172–186. arXiv:1812.08008.

14.1 Software and repositories

41. StyleGAN-Human official implementation. <https://github.com/stylegan-human/StyleGAN-Human>
42. ReStyle encoder official implementation. <https://github.com/yuval-alaluf/restyle-encoder>
43. PTI official implementation. <https://github.com/danielroich/PTI>
44. InsetGAN official repository. <https://github.com/afruehstueck/insetGAN>
45. This project: Whole-Body GAN Generator, built at Fellowship.AI, May–August 2022. Successor project: `whole-body-diffusion-generator` (2026), IP-Adapter FaceID + SDXL on Modal.